



Python Programming - 2301CS404

Lab - 1

Kalp Viradia | 23010101299 |
26/11/2024

01) WAP to print "Hello World"

```
In [2]: print('Hello World')
```

Hello World

02) WAP to print addition of two numbers with and without using input().

```
In [4]: a = int(input('Enter number 1: '))  
b = int(input('Enter number 2: '))  
print(a+b)  
  
c = 10  
d = 20  
print(c+d)
```

30
30

03) WAP to check the type of the variable.

```
In [6]: a = 10  
b = 10.5  
c = 'Hello'  
d = True  
print(type(a))  
print(type(b))  
print(type(c))  
print(type(d))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

04) WAP to calculate simple interest.

```
In [8]: principle = float(input('Enter the principle amount: '))
time = float(input('Enter the time period: '))
rate = float(input('Enter the rate of interest: '))
simpleInterest = (principle*time*rate)/100

print('Simple interest is:', simpleInterest)
```

Simple interest is: 1080.0

05) WAP to calculate area and perimeter of a circle.

```
In [10]: import math
r = float(input('Enter the radius of the circle: '))

perimeter = 2*math.pi*r
area = math.pi*r*r

print('Perimeter of the circle is:', perimeter)
print('Area of the circle is:', area)
```

Perimeter of the circle is: 94.24777960769379

Area of the circle is: 706.8583470577034

06) WAP to calculate area of a triangle.

```
In [12]: base = float(input('Enter the base of the triangle: '))
height = float(input('Enter the height of the triangle: '))
area = 1/2 * (base* height)

print('Area of the triangle is:', area)
```

Area of the triangle is: 90.0

07) WAP to compute quotient and remainder.

```
In [14]: a = int(input('Enter number 1: '))
b = int(input('Enter number 2: '))

print(f'Quotient is {a/b}')
print(f'Remainder is {a%b}')
```

Quotient is 0.4

Remainder is 2

08) WAP to convert degree into Fahrenheit and vice versa.

```
In [18]: temp = input('Enter "c" for celcius or "f" for fahrenheit: ')
if temp == 'c':
    c = float(input('Enter the temperature in celcius: '))
    f = c * (9/5) + 32
```

```

    print(f'{c} degrees celcius is equal to {f} degrees fahrenheit')
else:
    f = float(input('Enter the temperature in fahrenheit: '))
    c = (5/9) * (f-32)
    print(f'{f} degrees fahrenheit is equal to {c} degrees celcius')

```

32.0 degrees celcius is equal to 89.6 degrees fahrenheit

09) WAP to find the distance between two points in 2-D space.

```

In [20]: x1, x2 = 1, 4
        y1, y2 = 2, 6

        distance = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
        print(f'Distance between 2 points in 2-D space is {distance}')

```

Distance between 2 points in 2-D space is 5.0

10) WAP to print sum of n natural numbers.

```

In [22]: n = int(input('Enter any natural number: '))
        sum = 0
        for i in range(n+1):
            sum += i

        print(f'Sum of first {n} natural numbers is {sum}')

```

Sum of first 5 natural numbers is 15

11) WAP to print sum of square of n natural numbers.

```

In [24]: n = int(input('Enter any natural number: '))
        sum = 0
        for i in range(n+1):
            sum += (i*i)

        print(f'Sum of square of first {n} natural numbers is {sum}')

```

Sum of square of first 5 natural numbers is 55

12) WAP to concate the first and last name of the student.

```

In [26]: first = input('Enter your first name: ')
        last = input('Enter your last name: ')
        name = first + ' ' + last
        print(f'Your full name is {name}')

```

Your full name is Hello World

13) WAP to swap two numbers.

```

In [28]: a, b = 5, 10
        print(f'a is {a} and b is {b}')
        a, b = b, a
        print(f'a is {a} and b is {b}')

```

a is 5 and b is 10
a is 10 and b is 5

14) WAP to get the distance from user into kilometer, and convert it into meter, feet, inches and centimeter.

```
In [30]: distance = float(input('Enter distance in kilometer: '))
print(f'The distance in meter is {distance * 1000} meters')
print(f'The distance in feet is {distance * 3280.84} feet')
print(f'The distance in inches is {distance * 39370.08} inches')
print(f'The distance in centimeters is {distance * 100000} centimeters')
```

The distance in meter is 1500000.0 meters
The distance in feet is 4921260.0 feet
The distance in inches is 59055120.0 inches
The distance in centimeters is 150000000.0 centimeters

15) WAP to get day, month and year from the user and print the date in the given format: 23-11-2024.

```
In [32]: day = int(input('Enter the day: '))
month = int(input('Enter the month: '))
year = int(input('Enter the year: '))

print(f'The date is {day}-{month}-{year}')
```

The date is 23-11-2024

In []:



Python Programming - 2301CS404

Lab - 2

Kalp Viradia | 23010101299 |
03/12/2024

if..else..

01) WAP to check whether the given number is positive or negative.

```
In [4]: n = int(input('Enter a number: '))

if n == 0:
    print('Number is Zero')
elif n > 0:
    print('Number is Positive')
else:
    print('Number is Negative')
```

Number is Positive

02) WAP to check whether the given number is odd or even.

```
In [3]: n = int(input('Enter a number: '))

if (n%2 == 0):
    print('Number is Even')
else:
    print('Number is Odd')
```

Number is Odd

03) WAP to find out largest number from given two numbers using simple if and ternary operator.

```
In [6]: a = int(input('Enter the first number: '))
b = int(input('Enter the second number: '))

if a > b:
    print('First number is largest')
else:
    print('Second number is largest')

p = "First number is largest" if (a > b) else "Second number is largest"
print(p)
```

Second number is largest
y is greater than x

04) WAP to find out largest number from given three numbers.

```
In [5]: a = int(input('Enter the first number: '))
b = int(input('Enter the second number: '))
c = int(input('Enter the third number: '))

if (a > b and a > c):
    print('First number is largest')
elif (b > a and b > c):
    print('Second number is largest')
else:
    print('Third number is largest')
```

Third number is largest

05) WAP to check whether the given year is leap year or not.

[If a year can be divisible by 4 but not divisible by 100 then it is leap year but if it is divisible by 400 then it is leap year]

```
In [10]: year = int(input('Enter the year: '))

if ((year%4 == 0 and year%100 != 0) or year%400 != 0):
    print(f'{year} is Leap year')
else:
    print(f'{year} is not Leap year')
```

2024 is Leap year

06) WAP in python to display the name of the day according to the number given by the user.

```
In [14]: day = int(input('Enter the number of day: '))

match day:
    case 1:
        print('Today is monday')
    case 2:
        print('Today is tuesday')
    case 3:
        print('Today is wednesday')
```

```

case 4:
    print('Today is thusday')
case 5:
    print('Today is friday')
case 6:
    print('Today is saturday')
case 7:
    print('Today is sunday')
case _:
    print('Invalid number')

```

Today is friday

07) WAP to implement simple calculator which performs (add,sub,mul,div) of two no. based on user input.

```

In [18]: a = int(input('Enter the first number: '))
b = int(input('Enter the second number: '))
operator = input('Enter your operator: ')
match operator:
    case '+':
        print(f'Addition of {a} and {b} is {a+b}')
    case '-':
        print(f'Substraction of {a} and {b} is {a-b}')
    case '*':
        print(f'Multiplication of {a} and {b} is {a*b}')
    case '/':
        print(f'Division of {a} and {b} is {a/b}')
    case _:
        print('Invalid operator')

```

Division of 55 and 99 is 0.5555555555555556

08) WAP to read marks of five subjects. Calculate percentage and print class accordingly.

Fail below 35 Pass Class between 35 to 45 Second Class between 45 to 60 First Class between 60 to 70 Distinction if more than 70

```

In [35]: sum = 0
for i in range(1,6):
    marks = int(input(f'Enter the marks of subject{i}: '))
    sum += marks
avg = sum/5

if avg < 35:
    print('Failed')
elif avg >= 35 and avg < 45:
    print('Pass')
elif avg >= 45 and avg < 60:
    print('Second class')
elif avg >= 60 and avg < 70:
    print('First class')
elif avg > 70:
    print('Distinction')

```

First class

09) Three sides of a triangle are entered through the keyboard, WAP to check whether the triangle is isosceles, equilateral, scalene or right-angled triangle.

```
In [7]: a = float(input("Enter the first side: "))
b = float(input("Enter the second side: "))
c = float(input("Enter the third side: "))

if a + b > c and a + c > b and b + c > a:
    if a == b == c:
        print("The triangle is Equilateral.")
    elif a == b or b == c or a == c:
        print("The triangle is Isosceles.")
    elif a**2 + b**2 == c**2 or a**2 + c**2 == b**2 or b**2 + c**2 == a**2:
        print("The triangle is Right-angled.")
    else:
        print("The triangle is Scalene.")
else:
    print("The given sides do not form a valid triangle.")
```

The triangle is Right-angled.

10) WAP to find the second largest number among three user input numbers.

```
In [29]: a = int(input('Enter the first number: '))
b = int(input('Enter the second number: '))
c = int(input('Enter the third number: '))

if (a > b and a < c):
    print('First number is second largest')
elif (b > a and b < c):
    print('Second number is second largest')
else:
    print('Third number is second largest')
```

Second number is second largest

11) WAP to calculate electricity bill based on following criteria. Which takes the unit from the user.

a. First 1 to 50 units – Rs. 2.60/unit b. Next 50 to 100 units – Rs. 3.25/unit c. Next 100 to 200 units – Rs. 5.26/unit d. above 200 units – Rs. 8.45/unit

```
In [37]: units = int(input('Enter the number of units: '))
case = 0
if units > 0 and units < 50:
    case = 1
elif units >= 50 and units < 100:
    case = 2
elif units >= 100 and units < 200:
    case = 3
elif units > 200:
    case = 4
rate = 0
match case:
```



```
case 1:
    rate = units * 2.6
    print(f'Your electricity bill is {rate}')
case 2:
    rate = (50 * 2.6) + ((units - 50) * 3.25)
    print(f'Your electricity bill is {rate}')
case 3:
    rate = (50 * 2.6) + (50 * 3.25) + ((units - 100) * 5.26)
    print(f'Your electricity bill is {rate}')
case 4:
    rate = (50 * 2.6) + (50 * 3.25) + (100 * 5.26) + ((units - 200) * 8.45)
    print(f'Your electricity bill is {rate}')
```

Your electricity bill is 195.0



Python Programming - 2301CS404

Lab - 3

Kalp Viradia | 23010101299 |
10/12/2024

for and while loop

01) WAP to print 1 to 10.

```
In [25]: print('Numbers from 1 to 10: ')\n         for i in range(1, 11):\n             print(i)
```

Numbers from 1 to 10:

1
2
3
4
5
6
7
8
9
10

02) WAP to print 1 to n.

```
In [27]: n = int(input('Enter the number: '))\n         print(f'Numbers from 1 to {n}: ')\n         for i in range(1, n+1):\n             print(i)
```

Numbers from 1 to 9:

1
2
3
4
5
6
7
8
9

03) WAP to print odd numbers between 1 to n.

```
In [1]: n = int(input('Enter the number: '))
print(f'Odd number from 1 to {n}: ')
for i in range(1, n+1): # or for i in range(1, n+1, 2): print(i)
    if i%2 != 0:
        print(i)
```

Odd number from 1 to 9:

1
3
5
7
9

04) WAP to print numbers between two given numbers which is divisible by 2 but not divisible by 3.

```
In [31]: n = int(input('Enter the first number: '))
m = int(input('Enter the second number: '))
print(f'Numbers which is divisible by 2 but not divisible by 3 from {n} to {m}:')
for i in range(n, m+1):
    if i%2 == 0 and i%3 != 0:
        print(i)
```

Numbers which is divisible by 2 but not divisible by 3 from 5 to 21:

8
10
14
16
20

05) WAP to print sum of 1 to n numbers.

```
In [33]: n = int(input('Enter the number: '))
sum = 0
for i in range(1, n+1):
    sum += i
print(f'The sum of 1 to {n} is {sum}')
```

The sum of 1 to 9 is 45

06) WAP to print sum of series 1 + 4 + 9 + 16 + 25 + 36 + ...n.

```
In [43]: n = int(input('Enter the number: '))
sum = 0
for i in range(1, n+1):
    sum += (i**2)
print(f'The sum of square of 1 to {n} is {sum}')
```

The sum of square of 1 to 5 is 55

07) WAP to print sum of series $1 - 2 + 3 - 4 + 5 - 6 + 7 \dots n$.

```
In [45]: n = int(input('Enter the number: '))
sum = 0
for i in range(1, n+1):
    if i%2 == 0:
        sum -= i
    else:
        sum += i
print(f'The sum of series of 1 to {n} is {sum}')
```

The sum of series of 1 to 9 is 5

08) WAP to print multiplication table of given number.

```
In [53]: n = int(input('Enter the number: '))
print(f'The multiplication table for {n} is: ')
for i in range(1, 11):
    print(f'{n} X {i} = {n*i}')
```

The multiplication table for 5 is:

```
5 X 1 = 5
5 X 2 = 10
5 X 3 = 15
5 X 4 = 20
5 X 5 = 25
5 X 6 = 30
5 X 7 = 35
5 X 8 = 40
5 X 9 = 45
5 X 10 = 50
```

09) WAP to find factorial of the given number.

```
In [55]: n = int(input('Enter the number: '))
fac = 1
for i in range(1, n+1):
    fac *= i
print(f'The factorial of {n} is: {fac}')
```

The factorial of 5 is: 120

10) WAP to find factors of the given number.

```
In [57]: n = int(input('Enter the number: '))
print(f'The factor of {n} are: ')
for i in range(1, n+1):
    if n%i == 0:
        print(i)
```

The factor of 6 are:

1
2
3
6

11) WAP to find whether the given number is prime or not.

```
In [5]: n = int(input('Enter the number: '))
if n <= 1:
    print('The number is not prime')
else:
    count = 0
    for i in range(2, n): # or for i in range(2, int(math.sqrt(n)) + 1) also in=
        if n%i == 0:
            count += 1
    if count == 0:
        print('The number is prime')
    else:
        print('The number is not prime')
```

The number is not prime

12) WAP to print sum of digits of given number.

```
In [85]: n = int(input('Enter the number: '))
sum = 0
while n > 0:
    sum += n % 10
    n = n // 10
print(f"The sum of the digits is {sum}")
```

The sum of the digits is 16

13) WAP to check whether the given number is palindrome or not

```
In [13]: n = input('Enter the number: ')

if n == n[::-1]:
    print('The number is a palindrome')
else:
    print('The number is not a palindrome')
```

The number is a palindrome

14) WAP to print GCD of given two numbers.

```
In [2]: a = int(input("Enter the first number: "))
b = int(input("Enter the second number: "))

# Euclidean algorithm
while b != 0:
    a, b = b, a % b

print(f"The GCD of {a} and {b} is: {a}")
```

The GCD of 4 and 0 is: 4

In []:



Python Programming - 2301CS404

Lab - 4

Kalp Viradia | 23010101299 |
16/12/2024

String

01) WAP to check whether the given string is palindrome or not.

```
In [2]: string = input('Enter a string: ')
reverse = string[::-1]

if string == reverse:
    print('The string is a palindrome')
else:
    print('The string is not a palindrome')
```

The string is a palindrome

02) WAP to reverse the words in the given string.

```
In [23]: string = input('Enter a string: ')
reverseWords = ' '.join(string.split()[::-1])
reverse = ''.join(reversed(string))
reverseAtPosition = ' '.join(reversed(reverse.split()))

print(reverseWords)
print(reverseAtPosition)
print(reverse)
```

World Hello
olleH dlrow
dlrow olleH

03) WAP to remove ith character from given string.

```
In [12]: string = input('Enter a string: ')
i = int(input('Enter the position to remove a character: '))

if i < 0 or i >= len(string):
    print('Index out of range')
else:
    string = string[:i] + string[i+1:]
    print('Modified string:', string)
```

Modified string: hello world

04) WAP to find length of string without using len function.

```
In [14]: string = input('Enter the string: ')
length = 0

for i in string:
    length += 1

print(f'The length of the string is {length}')
```

The length of the string is 5

05) WAP to print even length word in string.

```
In [22]: string = input('Enter the string: ')
words = string.split()

for word in words:
    if len(word) % 2 == 0:
        print(word)
```

byee

06) WAP to count numbers of vowels in given string.

```
In [24]: string = input('Enter the string: ')
count = 0

for i in string:
    if (i == 'a' or i == 'e' or i == 'i' or i == 'o' or i == 'u'):
        count += 1

print(f'The number of vowels in the string is {count}')
```

The number of vowels in the string is 5

07) WAP to capitalize the first and last character of each word in a string.

```
In [26]: string = input("Enter a string: ")
words = string.split()
result = []

for word in words:
    if len(word) > 1:
        word = word[0].upper() + word[1:-1] + word[-1].upper()
    result.append(word)
```



```

else:
    word = word.upper()
    result.append(word)

output = ' '.join(result)
print("Modified string:", output)

```

Modified string: First Second

08) WAP to convert given array to string.

```

In [30]: array = ['Hello', 'World', 'From', 'Python']
         result = ' '.join(array)

         print("Converted string:", result)

```

Converted string: Hello World From Python

09) Check if the password and confirm password is same or not.

In case of only case's mistake, show the error message.

```

In [32]: password = input("Enter the password: ")
         confirm_password = input("Enter the confirm password: ")
         if password == confirm_password:
             print("Passwords match!")
         elif password.lower() == confirm_password.lower():
             print("Passwords do not match due to case differences.")
         else:
             print("Passwords do not match.")

```

Passwords do not match due to case differences.

10) : Display credit card number.

card no. : 1234 5678 9012 3456

display as : **** * 3456

```

In [75]: card_number = input("Enter the credit card number (with spaces): ")

         if ''.join(card_number.split()).isnumeric() and len(card_number) == 19:
             parts = card_number.split()
             masked_parts = ['****'] * (len(parts) - 1) + [parts[-1]]
             masked_card = ' '.join(masked_parts)
             print("Masked card number:", masked_card)
         else:
             print("Invalid card number")

```

Masked card number: **** * 3456

11) : Checking if the two strings are Anagram or not.

s1 = decimal and s2 = medical are Anagram

```
In [38]: s1 = input("Enter the first string: ")
s2 = input("Enter the second string: ")

s1 = s1.replace(" ", "").lower()
s2 = s2.replace(" ", "").lower()

if sorted(s1) == sorted(s2):
    print(f"{s1}" and "{s2}" are Anagrams.')
else:
    print(f"{s1}" and "{s2}" are not Anagrams.')
```

"decimal" and "medical" are Anagrams.

12) : Rearrange the given string. First lowercase then uppercase alphabets.

input : EHlsarwiwhtwMV

output : lsarwiwhtwEHMV

```
In [42]: string = input("Enter the string: ")

lowercase = ''.join([char for char in string if char.islower()])
uppercase = ''.join([char for char in string if char.isupper()])
result = lowercase + uppercase

print("Rearranged string:", result)
```

Rearranged string: lsarwiwhtwEHMV



Python Programming - 2301CS404

Lab - 5

Kalp Viradia | 23010101299 |
23/12/2024

List

01) WAP to find sum of all the elements in a List.

```
In [40]: list = []
          terminate = True
          while terminate == True:
              element = input('Enter a element for list (Enter "f" to terminate): ')
              if element == 'f':
                  break
              else:
                  element = int(element)
                  list.append(element)

          sum = 0
          for i in list:
              sum += i

          print(f'The sum of the list is: {sum}')
```

The sum of the list is: 30

02) WAP to find largest element in a List.

```
In [22]: list = []
          terminate = True
          while terminate == True:
              element = input('Enter a element for list (Enter "f" to terminate): ')
              if element == 'f':
                  break
              else:
                  element = int(element)
```

```

list.append(element)

largest = list[1]
for i in list:
    if i > largest:
        largest = i

print(f'The largest number is: {largest}')
```

The largest number is: 8

03) WAP to find the length of a List.

```

In [24]: list = []
         terminate = True
         while terminate == True:
             element = input('Enter a element for list (Enter "f" to terminate): ')
             if element == 'f':
                 break
             else:
                 element = int(element)
                 list.append(element)

         print(f'The length of the list is: {len(list)}')
```

The length of the list is: 9

04) WAP to interchange first and last elements in a list.

```

In [26]: list = []
         terminate = True
         while terminate == True:
             element = input('Enter a element for list (Enter "f" to terminate): ')
             if element == 'f':
                 break
             else:
                 element = int(element)
                 list.append(element)

         list[0], list[-1] = list[-1], list[0]
         print(f'List after interchange is: {list}')
```

List after interchange is: [2, 7, 8, 2, 4, 1, 5]

05) WAP to split the List into two parts and append the first part to the end.

```

In [28]: list = []
         terminate = True
         while terminate == True:
             element = input('Enter a element for list (Enter "f" to terminate): ')
             if element == 'f':
                 break
             else:
                 element = int(element)
                 list.append(element)

         # splitIndex = int(len(list)/2)
```

```
splitIndex = int(input('Enter the split index: '))
first = list[:splitIndex]
second = list[splitIndex:]

print(f'The list after split is: {second + first}')
```

The list after split is: [3, 1, 5, 4, 5, 7, 8, 9, 5]

06) WAP to interchange the elements on two positions entered by a user.

```
In [79]: list = []
terminate = True
while terminate == True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list.append(element)

position1 = int(input('Enter the position of the first element: '))
position2 = int(input('Enter the position of the second element: '))

if position1 > len(list) or position2 > len(list):
    print('Invalid position')
else:
    list[position1], list[position2] = list[position2], list[position1]
    print(f'List after interchange is: {list}')
```

List after interchange is: [4, 5, 6, 3, 77, 1, 2, 55]

07) WAP to reverse the list entered by user.

```
In [32]: list = []
terminate = True
while terminate == True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list.append(element)

reversedList = list[::-1]
print(f'The reversed of the list is: {reversedList}')
```

The reversed of the list is: [4, 5, 6, 7, 5]

08) WAP to print even numbers in a list.

```
In [34]: list = []
terminate = True
while terminate == True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
```

```
        element = int(element)
        list.append(element)

evenList = []
for i in list:
    if i % 2 == 0:
        evenList.append(i)

print(f'The length of the list is: {evenList}')
```

The length of the list is: [8, 6, 2]

09) WAP to count unique items in a list.

```
In [38]: list = []
        terminate = True
        while terminate == True:
            element = input('Enter a element for list (Enter "f" to terminate): ')
            if element == 'f':
                break
            else:
                element = int(element)
                list.append(element)

        uniqueItems = set(list)
        print(f'The number of unique elements in list are: {len(uniqueItems)}')
```

The number of unique elements in list are: 5

10) WAP to copy a list.

```
In [46]: list = []
        terminate = True
        while terminate == True:
            element = input('Enter a element for list (Enter "f" to terminate): ')
            if element == 'f':
                break
            else:
                element = int(element)
                list.append(element)

        copiedList = list.copy()
        print(f'The copied list is {copiedList}')
```

The copied list is [8, 4, 5, 6, 1, 2]

11) WAP to print all odd numbers in a given range.

```
In [54]: n = int(input('Enter the range: '))
        list = []

        for i in range(n+1):
            if i % 2 != 0:
                list.append(i)

        print(f'The list with odd numbers till {n} is {list}')
```

The list with odd numbers till 9 is [1, 3, 5, 7, 9]

12) WAP to count occurrences of an element in a list.

```
In [48]: list = []
terminate = True
while terminate == True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list.append(element)

countElement = int(input('Enter the element to count: '))
print(f'The count of the element {countElement} is {list.count(countElement)}')
```

The count of the element 5 is 2

13) WAP to find second largest number in a list.

```
In [50]: list = []
terminate = True
while terminate == True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list.append(element)

uniqueList = sorted(set(list), reverse=True)
print(f'The second largest number is {uniqueList[1]}')
```

The second largest number is 23

14) WAP to extract elements with frequency greater than K.

```
In [54]: list = []
terminate = True
while terminate == True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list.append(element)

k = int(input('Enter the frequency: '))

frequencyDict = {}
for i in list:
    if i in frequencyDict:
        frequencyDict[i] += 1
    else:
        frequencyDict[i] = 1

result = [key for key, value in frequencyDict.items() if value > k]
print(f'The element that occurred more than {k} times are: {result}')
```

The element that occurred more than 2 times are: [2]

15) WAP to create a list of squared numbers from 0 to 9 with and without using List Comprehension.

```
In [5]: #With List Comprehension
list = [i**2 for i in range(10)]

#Without List Comprehension
list = []
for i in range(10):
    list.append(i**2)

print(f'The list with square fom 0 to 9 is: {list}')
```

The list with square fom 0 to 9 is: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

16) WAP to create a new list (fruit whose name starts with 'b') from the list of fruits given by user.

```
In [61]: fruit_list = []
terminate = True
while terminate == True:
    element = input('Enter a fruit for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    fruit_list.append(element)

b_fruits = [i for i in fruit_list if i.lower().startswith('b')]
print(f"Fruits whose names start with 'b': {b_fruits}")
```

Fruits whose names start with 'b': ['banana', 'blueberry', 'blackberry']

17) WAP to create a list of common elements from given two lists.

```
In [65]: list1 = []
terminate = True
while terminate == True:
    element = input('Enter a element for list1 (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list1.append(element)

list2 = []
terminate = True
while terminate == True:
    element = input('Enter a element for list2 (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        list2.append(element)
```



```
common = [element for element in list1 if element in list2]  
print(f'Enter common elements in both list are: {common}')
```

Enter common elements in both list are: [4, 5, 6, 2, 1]

In []:



Python Programming - 2301CS404

Lab - 6

Kalp Viradia | 23010101299 |
30/12/2024

Tuple

01) WAP to find sum of tuple elements.

```
In [7]: t1 = []
while True:
    element = input('Enter an element for the tuple (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        t1.append(element)
t1 = tuple(t1)

print(f'The sum of the tuple elements is: {sum(t1)}')
```

The sum of the tuple elements is: 21

02) WAP to find Maximum and Minimum K elements in a given tuple.

```
In [5]: t1 = []
while True:
    element = input('Enter an element for the tuple (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        t1.append(element)
t1 = tuple(t1)

k = int(input("Enter the value of K: "))
```

```
sorted_t1 = sorted(t1)
minElements = sorted_t1[:k]
maxElements = sorted_t1[-k:]

print(f"The minimum {k} elements are: {minElements}")
print(f"The maximum {k} elements are: {maxElements}")
```

The minimum 2 elements are: [1, 2]

The maximum 2 elements are: [5, 6]

03) WAP to find tuples which have all elements divisible by K from a list of tuples.

```
In [22]: tuple_list = [(12, 24, 36), (5, 10, 15), (3, 6, 9), (4, 8, 12), (10, 20, 30)]
k = int(input("Enter the value of K: "))

result = []
for tup in tuple_list:
    if all(i % k == 0 for i in tup):
        result.append(tup)

print(f"Tuples with all elements divisible by {k} are: {result}")
```

Tuples with all elements divisible by 5 are: [(5, 10, 15), (10, 20, 30)]

04) WAP to create a list of tuples from given list having number and its cube in each tuple.

```
In [11]: l1 = []
while True:
    element = input('Enter a element for list (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        l1.append(element)

result = [(num, num**3) for num in l1]

print(f"List of tuples with number and its cube: {result}")
```

List of tuples with number and its cube: [(4, 64), (2, 8), (3, 27), (6, 216), (5, 125)]

05) WAP to find tuples with all positive elements from the given list of tuples.

```
In [ ]: tuple_list = [(12, 24, 36), (-5, 10, 15), (3, 6, 9), (4, -8, 12), (10, 20, 30)]

result = []
for tup in tuple_list:
    if all(i > 0 for i in tup):
        result.append(tup)

print(f"Tuples with all positive elements: {result}")
```

06) WAP to add tuple to list and vice – versa.

```
In [28]: l1 = [1, 2, 3, 4]
         t1 = (5, 6, 7)

         l1.append(t1)
         print("List after adding tuple:", l1)

         t2 = tuple(l1)
         t3 = t1 + tuple(l1)
         print("Tuple after adding list:", t3)
```

List after adding tuple: [1, 2, 3, 4, (5, 6, 7)]
 Tuple after adding list: (5, 6, 7, 1, 2, 3, 4, (5, 6, 7))

07) WAP to remove tuples of length K.

```
In [32]: tuple_list = [(1, 2, 3), (4, 5), (6, 7, 8, 9), (10, 11), (12, 13, 14)]
         k = int(input("Enter the value of K: "))

         filtered_list = [tup for tup in tuple_list if len(tup) != k]
         print(f"List after removing tuples of length {k}: {filtered_list}")
```

List after removing tuples of length 3: [(4, 5), (6, 7, 8, 9), (10, 11)]

08) WAP to remove duplicates from tuple.

```
In [13]: t1 = []
         while True:
             element = input('Enter an element for the tuple (Enter "f" to terminate): ')
             if element == 'f':
                 break
             else:
                 element = int(element)
                 t1.append(element)
         t1 = tuple(t1)

         unique_t1 = tuple(set(t1))
         print(f"Tuple after removing duplicates: {unique_t1}")
```

Tuple after removing duplicates: (1, 2, 3, 4, 5)

09) WAP to multiply adjacent elements of a tuple and print that resultant tuple.

```
In [15]: t1 = []
         while True:
             element = input('Enter an element for the tuple (Enter "f" to terminate): ')
             if element == 'f':
                 break
             else:
                 element = int(element)
                 t1.append(element)
         t1 = tuple(t1)

         result = []
```

```
for i in range(len(t1) - 1):
    product = t1[i] * t1[i + 1]
    result.append(product)
result_tuple = tuple(result)

print(f"Resultant tuple after multiplying adjacent elements: {result_tuple}")
```

Resultant tuple after multiplying adjacent elements: (8, 2, 3, 15, 330, 726)

10) WAP to test if the given tuple is distinct or not.

```
In [17]: t1 = []
while True:
    element = input('Enter an element for the tuple (Enter "f" to terminate): ')
    if element == 'f':
        break
    else:
        element = int(element)
        t1.append(element)
t1 = tuple(t1)

if len(t1) == len(set(t1)):
    print("The tuple is distinct.")
else:
    print("The tuple is not distinct.")
```

The tuple is distinct.



Python Programming - 2301CS404

Lab - 7

Kalp Viradia | 23010101299 |
12/01/2025

Set & Dictionary

01) WAP to iterate over a set.

```
In [2]: s1 = {1, 2, 3, 4, 5}

for i in s1:
    print(i)
```

1
2
3
4
5

02) WAP to convert set into list, string and tuple.

```
In [6]: s1 = {1, 2, 3, 4, 5}

l1 = list(s1)
print(f'List: {l1}')

str1 = ''.join(map(str, s1))
print(f'String: {str1}')

t1 = tuple(s1)
print(f'Tuple: {t1}')
```

List: [1, 2, 3, 4, 5]
String: 12345
Tuple: (1, 2, 3, 4, 5)

03) WAP to find Maximum and Minimum from a set.

```
In [8]: s1 = {1, 2, 3, 4, 5}

print(f'Maximum: {max(s1)}')
print(f'Minimum: {min(s1)}')
```

Maximum: 5

Minimum: 1

04) WAP to perform union of two sets.

```
In [10]: s1 = {1, 2, 3, 4, 5}
s2 = {3, 4, 5, 6, 7}

unionSet = s1 | s2 # or s1.union(s2)
print(unionSet)
```

{1, 2, 3, 4, 5, 6, 7}

05) WAP to check if two lists have at-least one element common.

```
In [16]: l1 = [1, 2, 3, 4]
l2 = [4, 5, 6, 7]

commonElements = set(l1) & set(l2)

if commonElements:
    print(f'The lists have at least one element in common: {commonElements}')
else:
    print('The lists do not have any common elements.')
```

The lists have at least one element in common: {4}

06) WAP to remove duplicates from list.

```
In [18]: l1 = [1, 1, 2, 3, 4, 5, 5, 6]

uniqueList = list(set(l1))
print(f'The list of unique elements is {uniqueList}')
```

The list of unique elements is [1, 2, 3, 4, 5, 6]

07) WAP to find unique words in the given string.

```
In [20]: str1 = "hello world Python world hello"

words = str1.lower().split()
uniqueWords = set(words)
print(f'The unique words are: {uniqueWords}')
```

The unique words are: {'world', 'hello', 'python'}

08) WAP to remove common elements of set A & B from set A.

```
In [22]: A = {1, 2, 3, 4, 5}
B = {3, 4, 6, 7}
A.difference_update(B)
print(f'Set A after removing common elements with B: {A}')
```

Set A after removing common elements with B: {1, 2, 5}

09) WAP to check whether two given strings are anagram or not using set.

```
In [24]: str1 = 'listen'
str2 = 'silent'

str1 = str1.replace(' ', '').lower()
str2 = str2.replace(' ', '').lower()

if set(str1) == set(str2) and len(str1) == len(str2):
    print(f"'{str1}' and '{str2}' are anagrams.")
else:
    print(f"'{str1}' and '{str2}' are not anagrams.")
```

'listen' and 'silent' are anagrams.

10) WAP to find common elements in three lists using set.

```
In [28]: list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]
list3 = [4, 5, 8, 9]

set1 = set(list1)
set2 = set(list2)
set3 = set(list3)

commonElements = set1 & set2 & set3
print(f'Common elements in the three lists are: {commonElements}')
```

Common elements in the three lists are: {4, 5}

11) WAP to count number of vowels in given string using set.

```
In [32]: str1 = 'Hello, how are you?'

vowels = {'a', 'e', 'i', 'o', 'u'}
vowelCount = 0
for char in str1.lower():
    if char in vowels:
        vowelCount += 1

print(f'Number of vowels in the string are: {vowelCount}')
```

Number of vowels in the string are: 7

12) WAP to check if a given string is binary string or not.


```
In [34]: str1 = '1010101'

if set(str1).issubset({'0', '1'}):
    print(f"'{str1}' is a binary string.")
else:
    print(f"'{str1}' is not a binary string.")
```

'1010101' is a binary string.

13) WAP to sort dictionary by key or value.

```
In [36]: d1 = {'apple': 5, 'banana': 2, 'cherry': 7, 'date': 1}

sortByKey = dict(sorted(d1.items()))
print(f'Dictionary sorted by key: {sortByKey}')

sortByValue = dict(sorted(d1.items(), key = lambda item: item[1]))
print(f'Dictionary sorted by value: {sortByValue}')
```

Dictionary sorted by key: {'apple': 5, 'banana': 2, 'cherry': 7, 'date': 1}

Dictionary sorted by value: {'date': 1, 'banana': 2, 'apple': 5, 'cherry': 7}

14) WAP to find the sum of all items (values) in a dictionary given by user. (Assume: values are numeric)

```
In [47]: d1 = {}

n = int(input('Enter the number of items in the dictionary: '))

for _ in range(n):
    key = input('Enter key: ')
    value = float(input(f'Enter value for {key}: '))
    d1[key] = value

totalSum = sum(d1.values())
print(f'The sum of all values in the dictionary is: {totalSum}')
```

The sum of all values in the dictionary is: 60.0

15) WAP to handle missing keys in dictionaries.

Example : Given, dict1 = {'a': 5, 'c': 8, 'e': 2}

if you look for key = 'd', the message given should be 'Key Not Found', otherwise print the value of 'd' in dict1.

```
In [49]: d1 = {'a': 5, 'c': 8, 'e': 2}

key = input('Enter the key to look for: ')

if key in d1:
    print(f"The value for key '{key}' is: {dict1[key]}")
else:
    print('Key Not Found')
```

Key Not Found

In []:



Python Programming - 2301CS404

Lab - 8

Kalp Viradia | 23010101299 |
12/01/2025

User Defined Function

01) Write a function to calculate BMI given mass and height.
($BMI = mass/h^{**2}$)

```
In [2]: def calculate_bmi(mass, height):  
        bmi = mass / (height ** 2)  
        return bmi  
  
        mass = 70  
        height = 1.75  
  
        bmi = calculate_bmi(mass, height)  
        print(f"BMI: {bmi}")
```

BMI: 22.857142857142858

02) Write a function that add first n numbers.

```
In [4]: def sum_of_first_n_numbers(n):  
        return n * (n + 1) // 2  
  
        n = 5  
        result = sum_of_first_n_numbers(n)  
        print(f"Sum of first {n} numbers: {result}")
```

Sum of first 5 numbers: 15

03) Write a function that returns 1 if the given number is Prime or 0 otherwise.

```
In [6]: def is_prime(number):
        if number <= 1:
            return 0
        for i in range(2, int(number ** 0.5) + 1):
            if number % i == 0:
                return 0
        return 1

num = 5
result = is_prime(num)
print(f"Is {num} prime? {result}")
```

Is 5 prime? 1

04) Write a function that returns the list of Prime numbers between given two numbers.

```
In [8]: def is_prime(number):
        if number <= 1:
            return False
        for i in range(2, int(number ** 0.5) + 1):
            if number % i == 0:
                return False
        return True

def primes_between(start, end):
    prime_numbers = []
    for num in range(start, end + 1):
        if is_prime(num):
            prime_numbers.append(num)
    return prime_numbers

start = 10
end = 50
prime_numbers = primes_between(start, end)
print(f"Prime numbers between {start} and {end}: {prime_numbers}")
```

Prime numbers between 10 and 50: [11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

05) Write a function that returns True if the given string is Palindrome or False otherwise.

```
In [10]: def is_palindrome(s):
        cleaned_s = ''.join(e for e in s.lower() if e.isalnum())
        return cleaned_s == cleaned_s[::-1]

s = "level"
result = is_palindrome(s)
print(f"Is the string '{s}' a palindrome? {result}")
```

Is the string 'level' a palindrome? True

06) Write a function that returns the sum of all the elements of the list.

```
In [12]: def sum_of_elements(lst):
        return sum(lst)
```

```
my_list = [1, 2, 3, 4, 5]
result = sum_of_elements(my_list)
print(result)
```

15

07) Write a function to calculate the sum of the first element of each tuples inside the list.

```
In [15]: def sum_first_elements(tuples_list):
          return sum(t[0] for t in tuples_list)

my_list = [(1, 2), (3, 4), (5, 6)]
result = sum_first_elements(my_list)
print(result)
```

9

08) Write a recursive function to find nth term of Fibonacci Series.

```
In [19]: def fibonacci(n):
          if n <= 1:
              return n
          else:
              return fibonacci(n - 1) + fibonacci(n - 2)

n = 5
result = fibonacci(n)
print(result)
```

5

09) Write a function to get the name of the student based on the given rollno.

Example: Given dict1 = {101:'Ajay', 102:'Rahul', 103:'Jay', 104:'Pooja'} find name of student whose rollno = 103

```
In [21]: def get_student_name(rollno, student_dict):
          return student_dict.get(rollno, "Roll number not found")

dict1 = {101: 'Ajay', 102: 'Rahul', 103: 'Jay', 104: 'Pooja'}
rollno = 103
student_name = get_student_name(rollno, dict1)
print(student_name)
```

Jay

10) Write a function to get the sum of the scores ending with zero.

Example : scores = [200, 456, 300, 100, 234, 678]

Ans = 200 + 300 + 100 = 600

```
In [23]: def sum_scores_ending_with_zero(scores):
          return sum(score for score in scores if score % 10 == 0)

scores = [200, 456, 300, 100, 234, 678]
result = sum_scores_ending_with_zero(scores)
print(result)
```

600

11) Write a function to invert a given Dictionary.

hint: keys to values & values to keys

Before : {'a': 10, 'b':20, 'c':30, 'd':40}

After : {10:'a', 20:'b', 30:'c', 40:'d'}

```
In [25]: def invert_dictionary(original_dict):
          return {v: k for k, v in original_dict.items()}

original_dict = {'a': 10, 'b': 20, 'c': 30, 'd': 40}
inverted_dict = invert_dictionary(original_dict)
print(inverted_dict)
```

```
{10: 'a', 20: 'b', 30: 'c', 40: 'd'}
```

12) Write a function to check whether the given string is Pangram or not.

hint: Pangram is a string containing all the characters a-z atleast once.

"the quick brown fox jumps over the lazy dog" is a Pangram string.

```
In [27]: def is_pangram(string):
          string = string.lower()
          alphabet = set('abcdefghijklmnopqrstuvwxyz')
          return set(string) >= alphabet

sentence = "the quick brown fox jumps over the lazy dog"
result = is_pangram(sentence)
print(result)
```

True

13) Write a function that returns the number of uppercase and lowercase letters in the given string.

example : Input : s1 = AbcDEfgh ,Ouptput : no_upper = 3, no_lower = 5

```
In [29]: def count_upper_lower(s):
          no_upper = sum(1 for char in s if char.isupper())
          no_lower = sum(1 for char in s if char.islower())
          return no_upper, no_lower

s1 = "AbcDEfgh"
```

```
no_upper, no_lower = count_upper_lower(s1)
print(f"no_upper = {no_upper}, no_lower = {no_lower}")
```

```
no_upper = 3, no_lower = 5
```

14) Write a lambda function to get smallest number from the given two numbers.

```
In [31]: smallest = lambda a, b: a if a < b else b

num1 = 5
num2 = 8
result = smallest(num1, num2)
print(result)
```

```
5
```

15) For the given list of names of students, extract the names having more than 7 characters. Use filter().

```
In [33]: def filter_long_names(names):
        return list(filter(lambda name: len(name) > 7, names))

students = ["John", "Alexander", "Sophia", "Michael", "Emma", "Jonathan"]
long_names = filter_long_names(students)
print(long_names)
```

```
['Alexander', 'Jonathan']
```

16) For the given list of names of students, convert the first letter of all the names into uppercase. use map().

```
In [37]: def capitalize_names(names):
        return list(map(lambda name: name.capitalize(), names))

students = ["john", "alexander", "sophia", "michael", "emma", "jonathan"]
capitalized_names = capitalize_names(students)
print(capitalized_names)
```

```
['John', 'Alexander', 'Sophia', 'Michael', 'Emma', 'Jonathan']
```

17) Write udfs to call the functions with following types of arguments:

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Length Positional(*args*) & variable length Keyword Arguments (**kwargs*)
5. Keyword-Only & Positional Only Arguments

```
In [43]: # 1. Positional Arguments
def positional_args(arg1, arg2, arg3):
    print(f"arg1: {arg1}, arg2: {arg2}, arg3: {arg3}")

# 2. Keyword Arguments
def keyword_args(arg1, arg2, arg3):
```

```

    print(f"arg1: {arg1}, arg2: {arg2}, arg3: {arg3}")

# 3. Default Arguments
def default_args(arg1, arg2=20, arg3=30):
    print(f"arg1: {arg1}, arg2: {arg2}, arg3: {arg3}")

# 4. Variable Length Positional (*args) & Variable Length Keyword (**kwargs)
def variable_length_args(*args, **kwargs):
    print(f"Positional arguments: {args}")
    print(f"Keyword arguments: {kwargs}")

# 5. Keyword-Only and Positional-Only Arguments
def mixed_args(arg1, arg2, /, arg3, arg4, *, arg5):
    print(f"Positional-only arguments: {arg1}, {arg2}")
    print(f"Positional-or-keyword argument: {arg3}, {arg4}")
    print(f"Keyword-only argument: {arg5}")

positional_args(10, 20, 30)
keyword_args(arg1=10, arg2=20, arg3=30)
default_args(10)
default_args(10, 40)
variable_length_args(10, 20, 30, name="John", age=25)
mixed_args(10, 20, 30, 40, arg5=50)

```

```

arg1: 10, arg2: 20, arg3: 30
arg1: 10, arg2: 20, arg3: 30
arg1: 10, arg2: 20, arg3: 30
arg1: 10, arg2: 40, arg3: 30
Positional arguments: (10, 20, 30)
Keyword arguments: {'name': 'John', 'age': 25}
Positional-only arguments: 10, 20
Positional-or-keyword argument: 30, 40
Keyword-only argument: 50

```

In []:



Python Programming - 2301CS404

Lab - 9

Kalp Viradia | 23010101299 |
28/01/2025

File I/O

01) WAP to read and display the contents of a text file. (also try to open the file in some other directory)

- in the form of a string
- line by line
- in the form of a list

```
In [82]: fp = open('./Lab 9 Files/F1.txt')
print(fp.read(), end= '\n \n')
fp.seek(0,0)
print(fp.readline())
fp.seek(0,0)
print(fp.readlines())
fp.close()
```

```
Name, Age, Department
Alice, 23, Computer Science
Bob, 24, Electrical Engineering
Charlie, 22, Mechanical Engineering
Diana, 21, Civil Engineering
Eve, 23, Data Science
```

```
Name, Age, Department
```

```
['Name, Age, Department\n', 'Alice, 23, Computer Science\n', 'Bob, 24, Electrical Engineering\n', 'Charlie, 22, Mechanical Engineering\n', 'Diana, 21, Civil Engineering\n', 'Eve, 23, Data Science']
```

02) WAP to create file named "new.txt" only if it doesn't exist.

```
In [84]: fp = open('./Lab 9 Files/F2.txt', 'w')
fp.close()
```

03) WAP to read first 5 lines from the text file.

```
In [86]: fp = open('./Lab 9 Files/F3.txt')
for i in range(5):
    print(fp.readline())
fp.close()
```

Name, Age, Department

Alice, 23, Computer Science

Bob, 24, Electrical Engineering

Charlie, 22, Mechanical Engineering

Diana, 21, Civil Engineering

04) WAP to find the longest word(s) in a file

```
In [88]: fp = open('./Lab 9 Files/F3.txt')

words = list(fp.read().split())
words_len = list(map(len, words))
max_len = max(words_len)
ans = [i for i in words if len(i) == max_len]
print(ans)
fp.close()
```

['Engineering', 'Engineering', 'Engineering']

05) WAP to count the no. of lines, words and characters in a given text file.

```
In [2]: fp = open('./Lab 9 Files/F3.txt')

print(f'The number of lines in the file is: {len(fp.readlines())}')

fp.seek(0,0)
print(f'The number of words in the file is: {len(fp.read().split())}')

fp.seek(0,0)
print(f'The number of characters in the file is: {len(fp.read())}')
fp.close()
```

The number of lines in the file is: 6

The number of words in the file is: 23

The number of characters in the file is: 168

06) WAP to copy the content of a file to the another file.

```
In [10]: with open('./Lab 9 Files/F3.txt', 'r') as fp_read, open('./Lab 9 Files/F4.txt',
          fp_write.write(fp_read.read())

print("Content has been copied successfully!")
```

Content has been copied successfully!

07) WAP to find the size of the text file.

```
In [8]: fp = open('./Lab 9 Files/F3.txt')
print(f'The size of the file is {len(fp.read())} bytes.')
fp.close()
```

The size of the file is 168 bytes.

08) WAP to create an UDF named frequency to count occurrences of the specific word in a given text file.

```
In [18]: def frequency(word, filepath):
          with open(filepath) as fp:
              l1 = fp.read().split()
              count = l1.count(word)
              print(f'Count of the {word} is {count}.')

          frequency('Engineering', './Lab 9 Files/F3.txt')
```

Count of the Engineering is 3.

09) WAP to get the score of five subjects from the user, store them in a file. Fetch those marks and find the highest score.

```
In [9]: def get_scores():
          scores = []
          for i in range(1, 6):
              score = int(input(f"Enter the score for subject {i}: "))
              scores.append(score)

          with open('./Lab 9 Files/F5.txt', 'w') as fp:
              for score in scores:
                  fp.write(f"{score}\n")

          def find_highest_score():
              with open('./Lab 9 Files/F5.txt') as fp:
                  scores = [int(line.strip()) for line in fp.readlines()]

              highest_score = max(scores)
              print(f"The highest score is: {highest_score}")

          get_scores()
          find_highest_score()
```

The highest score is: 98

10) WAP to write first 100 prime numbers to a file named primenumbers.txt

(Note: each number should be in new line)

```
In [11]: def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

def write_prime_numbers_to_file():
    primes = []
    num = 2
    while len(primes) < 100:
        if is_prime(num):
            primes.append(num)
            num += 1

    with open('./Lab 9 Files/primenumbers.txt', 'w') as fp:
        for prime in primes:
            fp.write(f"{prime}\n")
    print('Added first 100 prime numbers to "primenumbers.txt".')

write_prime_numbers_to_file()
```

Added first 100 prime numbers to "primenumbers.txt".

11) WAP to merge two files and write it in a new file.

```
In [19]: def merge_files(file1, file2, merged_file):
    f1 = open(file1, 'r')
    f2 = open(file2, 'r')
    output = open(merged_file, 'w')

    output.write(f1.read())
    output.write("\n")
    output.write(f2.read())

    f1.close()
    f2.close()
    output.close()

    print(f"Contents of {file1} and {file2} have been merged into {merged_file}")

file1 = './Lab 9 Files/file1.txt'
file2 = './Lab 9 Files/file2.txt'
merged_file = './Lab 9 Files/merged_file.txt'

merge_files(file1, file2, merged_file)
```

Contents of ./Lab 9 Files/file1.txt and ./Lab 9 Files/file2.txt have been merged into ./Lab 9 Files/merged_file.txt

12) WAP to replace word1 by word2 of a text file. Write the updated data to new file.

```
In [17]: def replace_word_in_file(input_file, output_file, word1, word2):
    infile = open(input_file, 'r')
    content = infile.read()
    infile.close()

    updated_content = content.replace(word1, word2)

    outfile = open(output_file, 'w')
    outfile.write(updated_content)
    outfile.close()

    print(f"Replaced '{word1}' with '{word2}' and saved to {output_file}")

    input_file = './Lab 9 Files/input.txt'
    output_file = './Lab 9 Files/output.txt'
    word1 = 'oldword'
    word2 = 'newword'

    replace_word_in_file(input_file, output_file, word1, word2)
```

Replaced 'oldword' with 'newword' and saved to ./Lab 9 Files/output.txt

13) Demonstrate tell() and seek() for all the cases(seek from beginning-end-current position) taking a suitable example of your choice.

```
In [29]: with open('./Lab 9 Files/sample.txt', 'w') as fp:
    fp.write("Hello, this is a sample file for demonstrating tell() and seek().")

    with open('./Lab 9 Files/sample.txt', 'r') as fp:
        # Case 1: Tell the current position from the beginning
        print("Case 1: Current position from beginning (before reading):", fp.tell())
        print("Reading first 5 characters:")
        print(fp.read(5)) # Read first 5 characters
        print("Position after reading 5 characters:", fp.tell())

        # Case 2: Seek from the beginning (SEEK_SET)
        fp.seek(0, 0) # Move the pointer to the beginning
        print("\nCase 2: Seek from the beginning to position 0:", fp.tell())
```

Case 1: Current position from beginning (before reading): 0

Reading first 5 characters:

Hello

Position after reading 5 characters: 5

Case 2: Seek from the beginning to position 0: 0

In []:



Python Programming - 2301CS404

Lab - 10

Kalp Viradia | 23010101299 |
03/02/2025

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [3]: try:
        # ZeroDivisionError
        result = 10 / 0

        # ValueError
        value = int("not_a_number")

        # TypeError
        result = "string" + 10

    except ZeroDivisionError:
        print("Error: Cannot divide by zero.")
    except ValueError:
        print("Error: Invalid value provided.")
    except TypeError:
        print("Error: Type mismatch encountered.")

    try:
        # ZeroDivisionError
        result = 10 / 0
```

```

# ValueError
value = int("not_a_number")

# TypeError
result = "string" + 10

except (ZeroDivisionError, ValueError, TypeError) as e:
    print(f"Error: {str(e)}")

```

Error: Cannot divide by zero.

Error: division by zero

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

```

In [15]: # IndexError
try:
    my_list = [1, 2, 3]
    print(my_list[5])
except IndexError as e:
    print(f"IndexError: {str(e)}")

# KeyError
try:
    my_dict = {'a': 1, 'b': 2}
    print(my_dict['c'])
except KeyError as e:
    print(f"KeyError: {str(e)}")

```

IndexError: list index out of range

KeyError: 'c'

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```

In [21]: # FileNotFoundError
try:
    with open("non_existent_file.txt", "r") as file:
        content = file.read()

except FileNotFoundError as e:
    print(f"FileNotFoundError: {str(e)}")

# ModuleNotFoundError
try:
    import non_existent_module

except ModuleNotFoundError as e:
    print(f"ModuleNotFoundError: {str(e)}")

```

FileNotFoundError: [Errno 2] No such file or directory: 'non_existent_file.txt'

ModuleNotFoundError: No module named 'non_existent_module'

04) WAP that catches all type of exceptions in a single except block.

```
In [19]: try:
# ZeroDivisionError
result = 10 / 0

# IndexError
# my_list = [1, 2, 3]
# print(my_list[5])

# ValueError
# value = int("not_a_number")

# KeyError
# my_dict = {'a': 1}
# print(my_dict['b'])

# FileNotFoundError
# with open("non_existent_file.txt", "r") as file:
#     content = file.read()

# ModuleNotFoundError
# import non_existent_module

except Exception as e:
    print(f"An error occurred: {str(e)}")
```

An error occurred: division by zero

05) WAP to demonstrate else and finally block.

```
In [25]: try:
    result = 10 / 2
    print(result)

except ZeroDivisionError as e:
    print(f"An error occurred: {str(e)}")

else:
    print("No exceptions occurred. The try block executed successfully.")

finally:
    print("This block is always executed, regardless of exceptions.")
```

5.0

No exceptions occurred. The try block executed successfully.

This block is always executed, regardless of exceptions.

06) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```
In [31]: grades_input = input("Enter a list of grades separated by commas: ")

try:
    grades = [int(grade.strip()) for grade in grades_input.split(',')]
    print("Grades:", grades)

except ValueError:
    print("Error: Some of the values you entered cannot be converted to integers")
```

Error: Some of the values you entered cannot be converted to integers. Please enter valid numbers.

07) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```
In [33]: def divide(a, b):
    try:
        result = a / b

    except ZeroDivisionError:
        return "Error: Cannot divide by zero."

    else:
        return result

a = float(input("Enter the first number: "))
b = float(input("Enter the second number: "))

result = divide(a, b)
print(f"Result: {result}")
```

Result: Error: Cannot divide by zero.

08) WAP that gets an age of a person from the user and raises ValueError with error message: "Enter Valid Age" :

If the age is less than 18.

otherwise print the age.

```
In [35]: def get_age():
    try:
        age = int(input("Enter your age: "))

        if age < 18:
            raise ValueError("Enter Valid Age")

        else:
            print(f"Your age is {age}")

    except ValueError as e:
        print(f"Error: {str(e)}")

get_age()
```

Error: Enter Valid Age

09) WAP to raise your custom Exception named InvalidUsernameError with the error message : "Username must be between 5 and 15 characters long":

if the given name is having characters less than 5 or greater than 15.

otherwise print the given username.

```
In [39]: class InvalidUsernameError(Exception):
          pass

          def check_username():
              try:
                  username = input("Enter your username: ")

                  if len(username) < 5 or len(username) > 15:
                      raise InvalidUsernameError("Username must be between 5 and 15 charac

                  else:
                      print(f"Your username is: {username}")

              except InvalidUsernameError as e:
                  print(f"Error: {str(e)}")

          check_username()
```

Error: Username must be between 5 and 15 characters long

10) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```
In [47]: import math

          class NegativeNumberError(Exception):
              pass

          def calculate_square_root():
              try:
                  number = float(input("Enter a number to calculate its square root: "))

                  if number < 0:
                      raise NegativeNumberError("Cannot calculate the square root of a neg

                  sqrt_result = math.sqrt(number)
                  print(f"The square root of {number} is: {sqrt_result}")

              except NegativeNumberError as e:
                  print(f"Error: {str(e)}")

              except ValueError:
```

```
print("Error: Please enter a valid number.")  
calculate_square_root()
```

Error: Cannot calculate the square root of a negative number

In []:



Python Programming - 2301CS404

Lab - 11

Kalp Viradia | 23010101299 |
10/02/2025

Modules

01) WAP to create Calculator module which defines functions like add, sub, mul and div.

Create another .py file that uses the functions available in Calculator module.

```
In [3]: from cal import add, multiply, subtract, divide

print(add(1,2))
print(subtract(1,2))
print(multiply(1,2))
print(divide(1,2))
```

```
3
-1
2
0.5
```

02) WAP to pick a random character from a given String.

```
In [5]: import random
s = 'python'
print(random.choice(s))
```

```
h
```

03) WAP to pick a random element from a given list.

```
In [7]: import random
l1 = [1,2,3,4,5]
print(random.choice(l1))
```

2

04) WAP to roll a dice in such a way that every time you get the same number.

```
In [24]: import random

random.seed(1)

def roll_dice():
    return random.randint(1,6)

print(roll_dice())
```

2

05) WAP to generate 3 random integers between 100 and 999 which is divisible by 5.

```
In [17]: import random

def get_random_integer():
    count = 0
    l1 = []
    while count != 3:
        random_number = random.randint(100, 999)

        if random_number % 5 == 0:
            l1.append(random_number)
            count += 1

    return l1

print(get_random_integer())
```

[950, 525, 900]

06) WAP to generate 100 random lottery tickets and pick two lucky tickets from it and announce them as Winner and Runner up respectively.

```
In [26]: import random

tickets = [''.join(random.choices('0123456789', k = 6)) for _ in range(100)]

winner = random.choice(tickets)
tickets.remove(winner)
runner_up = random.choice(tickets)

print(f"Winner: Ticket {winner}")
print(f"Runner-up: Ticket {runner_up}")
```

Winner: Ticket 876020
Runner-up: Ticket 193596

07) WAP to print current date and time in Python.

```
In [28]: import datetime

current_time = datetime.datetime.now()

print(current_time)
```

2025-02-11 10:24:20.989901

08) Subtract a week (7 days) from a given date in Python.

```
In [30]: import datetime

new_date = datetime.datetime.now() - datetime.timedelta(weeks = 1)

print(f>Date after subtracting a week: {new_date}")
```

Date after subtracting a week: 2025-02-04 10:25:27.316088

09) WAP to Calculate number of days between two given dates.

```
In [34]: import datetime

start_date = datetime.date(2025, 2, 4)
end_date = datetime.date(2025, 2, 11)

no_of_days = abs(start_date - end_date)

print(no_of_days)
```

7 days, 0:00:00

10) WAP to Find the day of the week of a given date.(i.e. whether it is sunday/monday/tuesday/etc.)

```
In [36]: import datetime

given_date = datetime.date(2025, 2, 11)

day_name = given_date.strftime("%A")

print("Day Name:", day_name)
```

Day Name: Tuesday

11) WAP to demonstrate the use of date time module.

```
In [38]: import datetime

print(datetime.datetime.now())
```

2025-02-11 10:28:08.166593

12) WAP to demonstrate the use of the math module.

```
In [40]: import math

print(math.pow(5,2))
print(math.gcd(2,8))
print(math.sqrt(25))
print(math.factorial(5))
print(math.log(10))
print(math.pi)
print(math.e)
print(math.tan(math.pi/2))
print(math.sin(math.pi/2))
print(math.cos(math.pi/2))
print(math.ceil(3.7))
print(math.floor(3.7))
print(math.fabs(-3.7))
print(math.fmod(10,3))
print(math.trunc(3.7))
print(math.fsum([1,2,3,4,5]))
print(math.prod([1,2,3,4,5]))
print(math.gcd(12,18))
print(math.lcm(12,18))
print(math.perm(5,2))
print(math.gcd(12,18))
print(math.comb(5,2))
print(math.trunc(3.7))
print(math.fsum([1,2,3,4,5]))
print(math.prod([1,2,3,4,5]))
print(math.lcm(12,18))
```

```
25.0
2
5.0
120
2.302585092994046
3.141592653589793
2.718281828459045
1.633123935319537e+16
1.0
6.123233995736766e-17
4
3
3.7
1.0
3
15.0
120
6
36
20
6
10
3
15.0
120
36
```

In []:



Python Programming - 2301CS404

Lab - 12

Kalp Viradia | 23010101299 |
17/02/2025

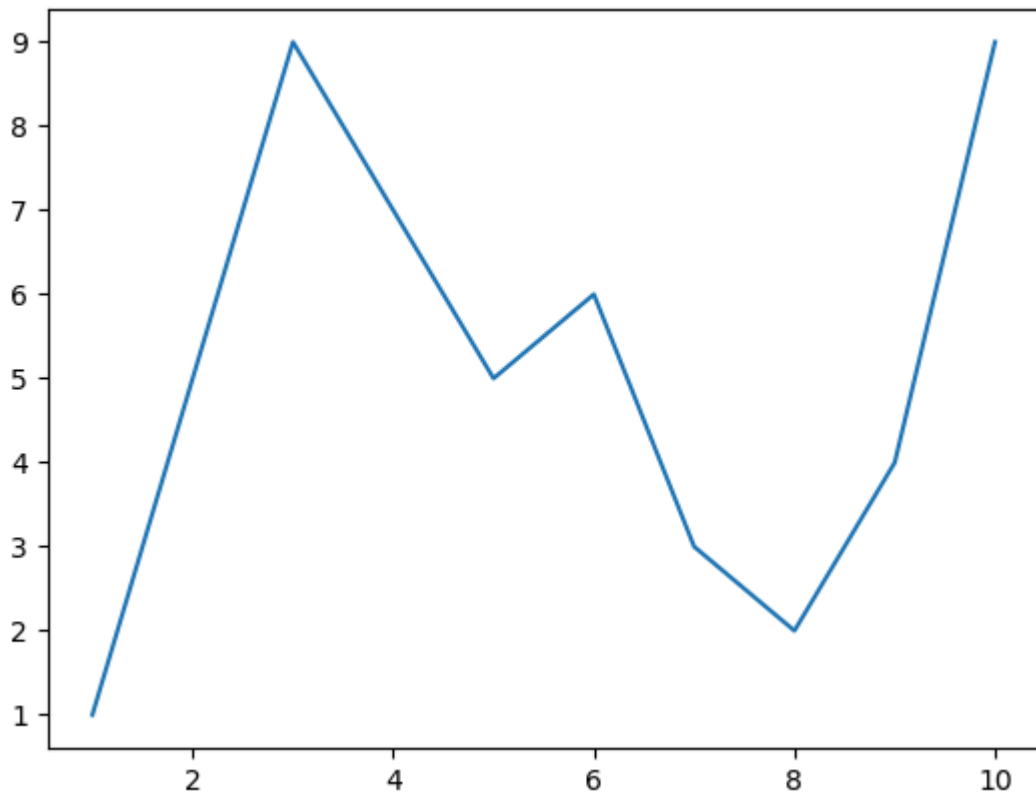
```
In [13]: #import matplotlib below

import matplotlib.pyplot as plt
import random as r
```

```
In [3]: x = range(1,11)
y = [1,5,9,7,5,6,3,2,4,9]

# write a code to display the line chart of above x & y

plt.plot(x, y)
# plt.title('Line Chart of x and y')
# plt.xlabel('x values')
# plt.ylabel('y values')
# plt.grid(True)
plt.show()
```



```
In [7]: x = [1,2,3,4,5,6,7,8,9,10]
cxMarks = [5,8,9,6,3,2,4,8,8,9]
cyMarks = [8,9,6,3,5,7,4,1,2,6]

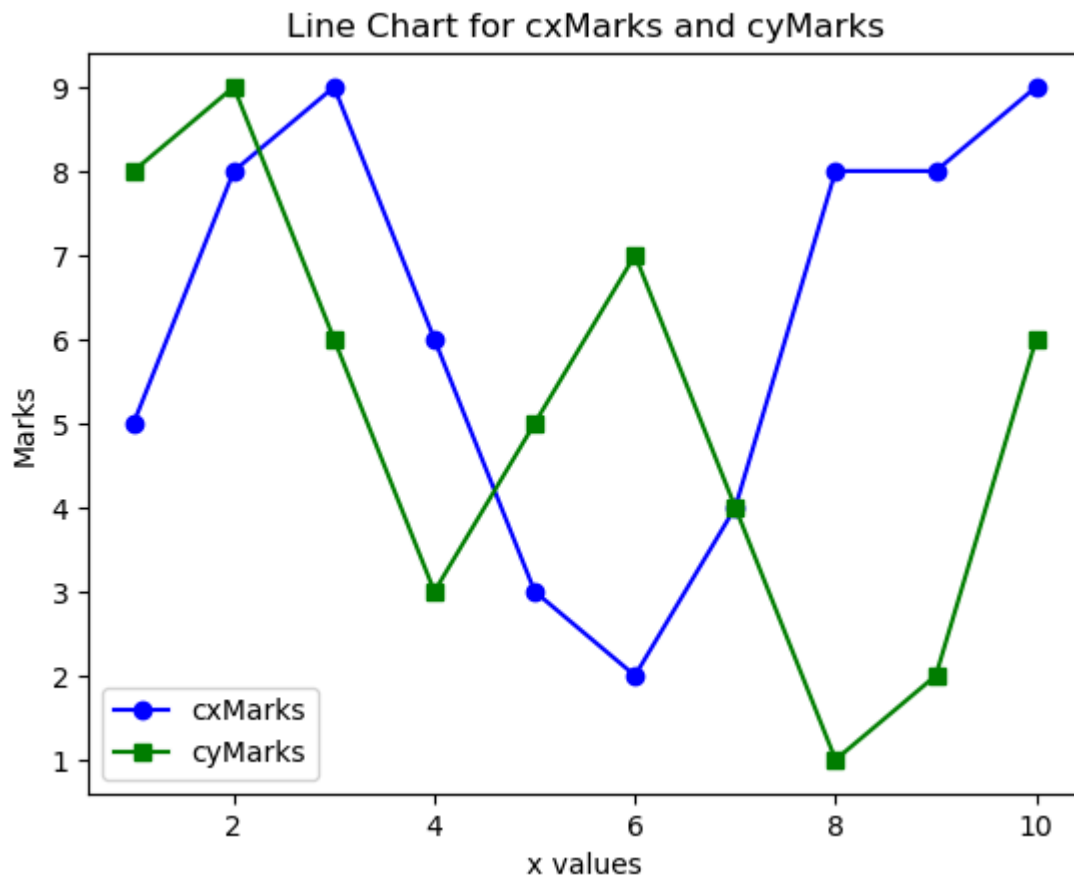
# write a code to display two lines in a line chart (data given above)

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
cxMarks = [5, 8, 9, 6, 3, 2, 4, 8, 8, 9]
cyMarks = [8, 9, 6, 3, 5, 7, 4, 1, 2, 6]

plt.plot(x, cxMarks, label='cxMarks', color='blue', marker='o')
plt.plot(x, cyMarks, label='cyMarks', color='green', marker='s')

plt.title('Line Chart for cxMarks and cyMarks')
plt.xlabel('x values')
plt.ylabel('Marks')
# plt.grid(True)
plt.legend()

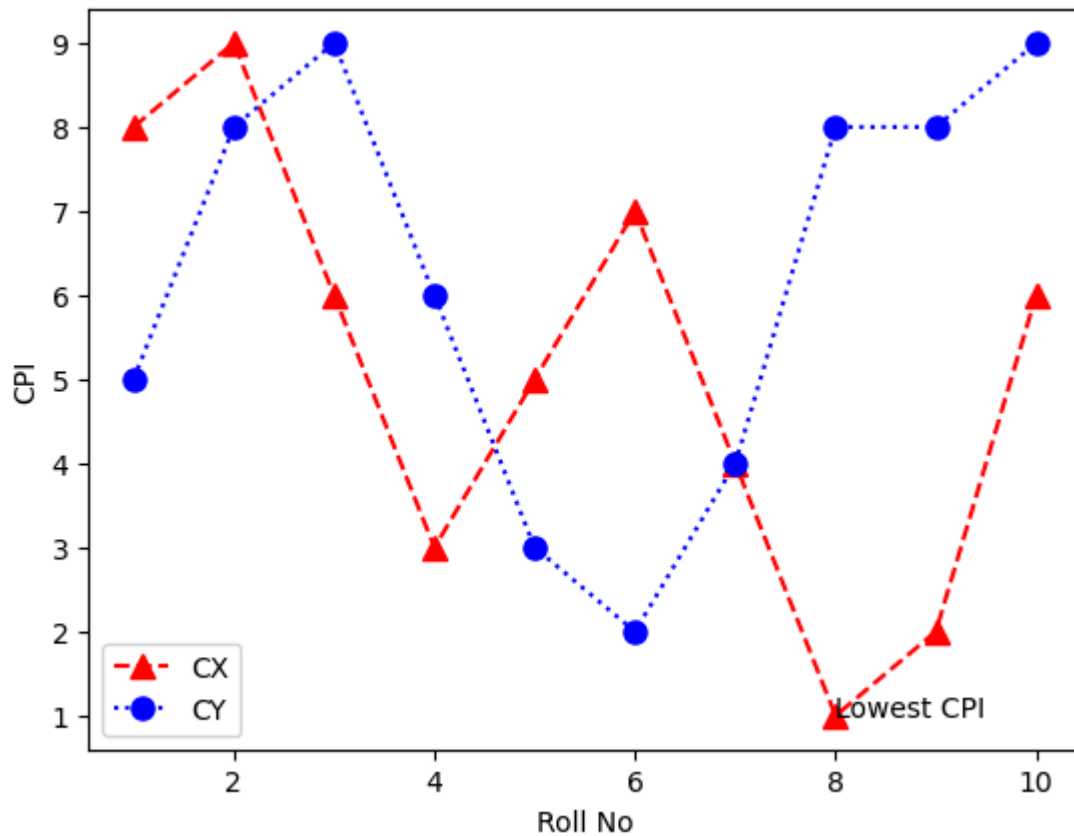
plt.show()
```



```
In [32]: x = range(1,11,1)
cxMarks= [8,9,6,3,5,7,4,1,2,6]
cyMarks= [5,8,9,6,3,2,4,8,8,9]

# write a code to generate below graph

plt.plot(x, cxMarks, 'r--^', label='CX', markersize=8)
plt.plot(x, cyMarks, 'b:o', label='CY', markersize=8)
plt.annotate('Lowest CPI', xy=(8, 1), fontsize=10)
plt.xlabel("Roll No")
plt.ylabel("CPI")
plt.legend()
plt.show()
```

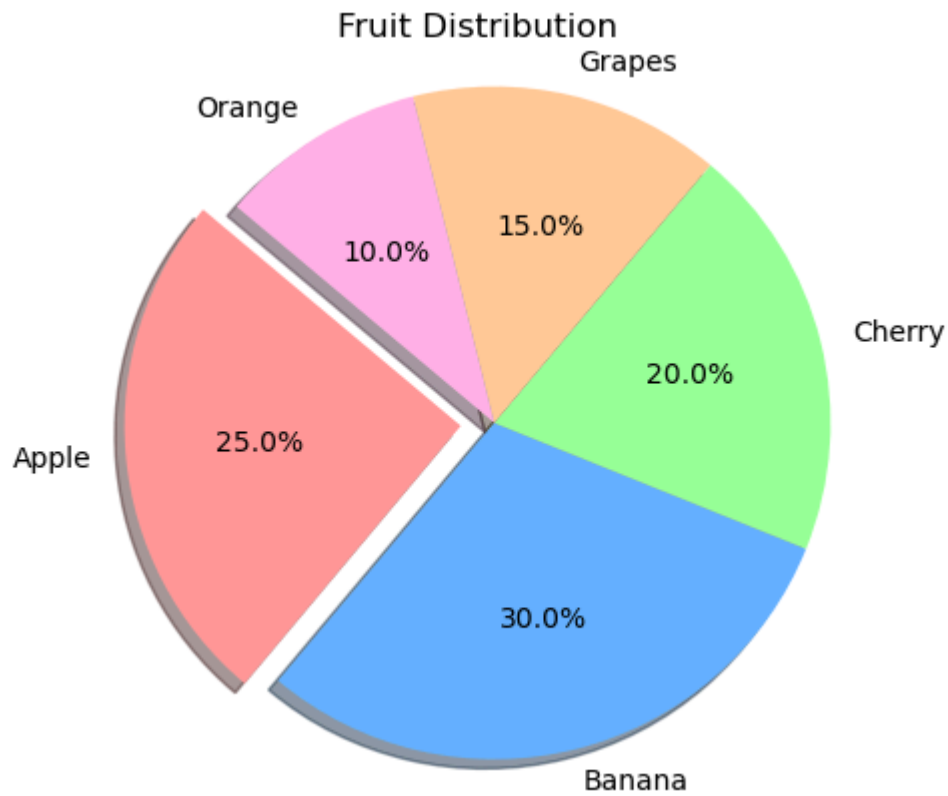


In []:

04) WAP to demonstrate the use of Pie chart.

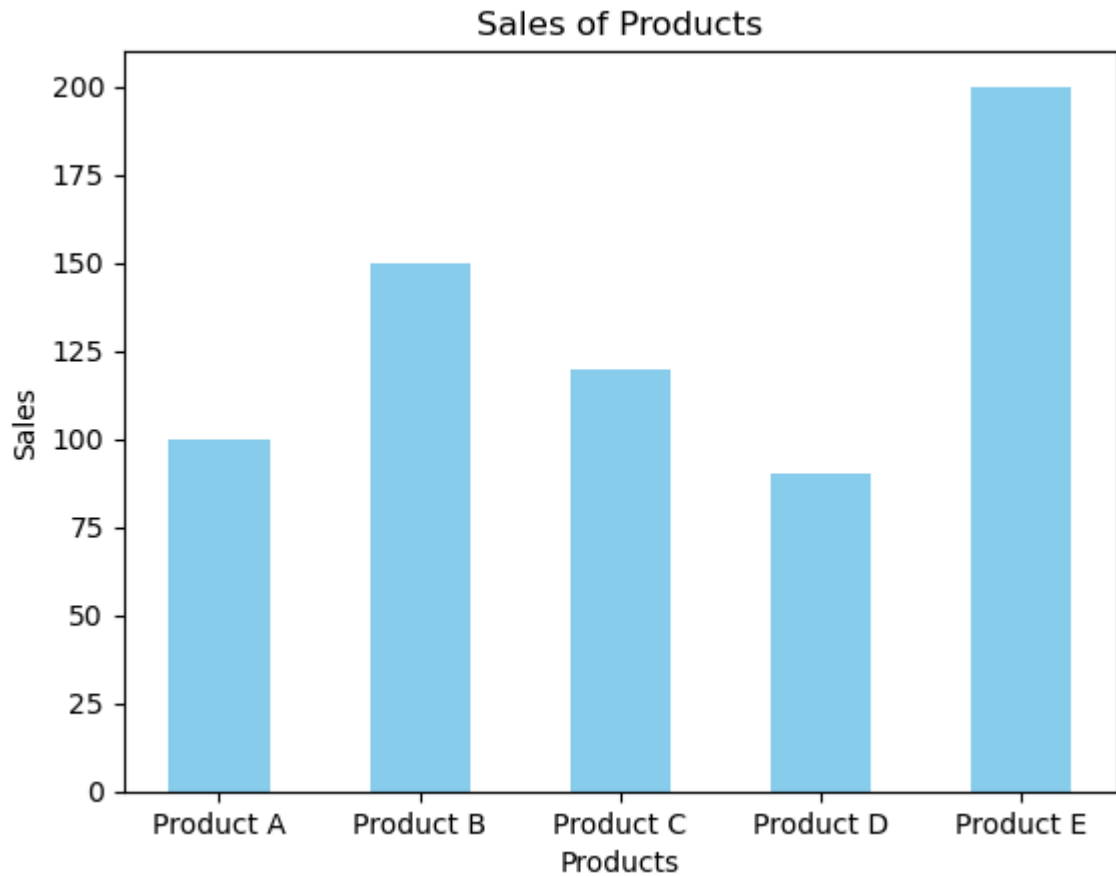
```
In [19]: labels = ['Apple', 'Banana', 'Cherry', 'Grapes', 'Orange']
        sizes = [25, 30, 20, 15, 10]
        colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99', '#ffb3e6']
        explode = (0.1, 0, 0, 0, 0)

        plt.pie(sizes, labels=labels, colors=colors, explode=explode, autopct='%1.1f%%',
        plt.axis('equal')
        plt.title('Fruit Distribution')
        plt.show()
```



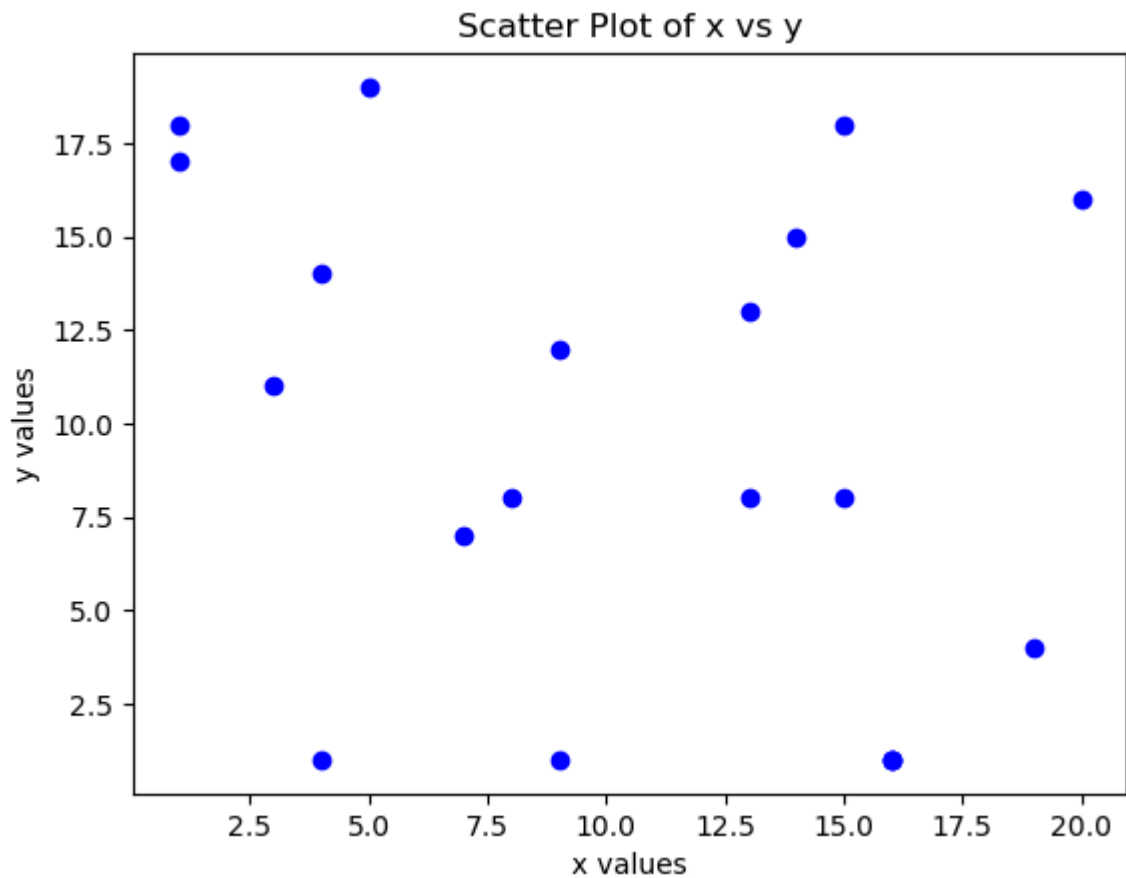
05) WAP to demonstrate the use of Bar chart.

```
In [21]: products = ['Product A', 'Product B', 'Product C', 'Product D', 'Product E']
sales = [100, 150, 120, 90, 200]
plt.bar(products, sales, color='skyblue', width = 0.5)
plt.title('Sales of Products')
plt.xlabel('Products')
plt.ylabel('Sales')
plt.show()
```



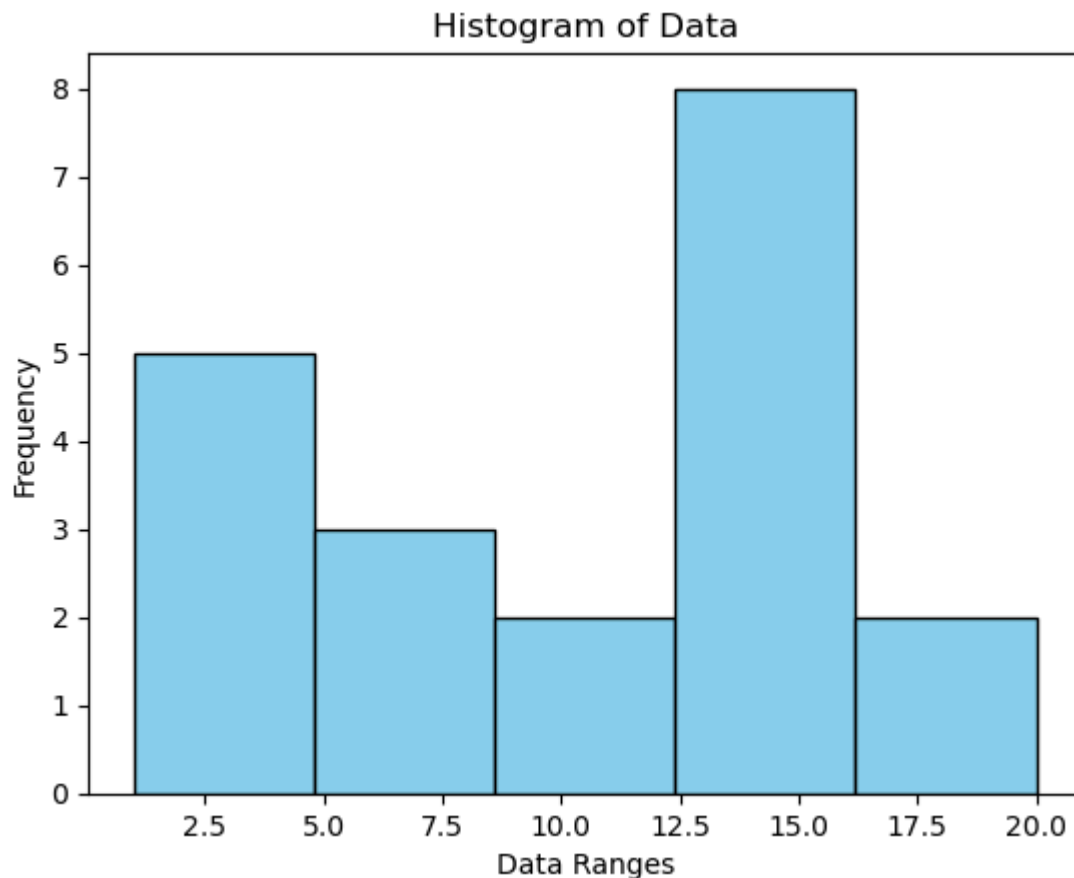
06) WAP to demonstrate the use of Scatter Plot.

```
In [25]: r.seed(1)
x = [r.randint(1,20) for i in range(20)]
y = [r.randint(1,20) for i in range(20)]
plt.scatter(x, y, color='blue', marker='o')
plt.title('Scatter Plot of x vs y')
plt.xlabel('x values')
plt.ylabel('y values')
plt.show()
```



07) WAP to demonstrate the use of Histogram.

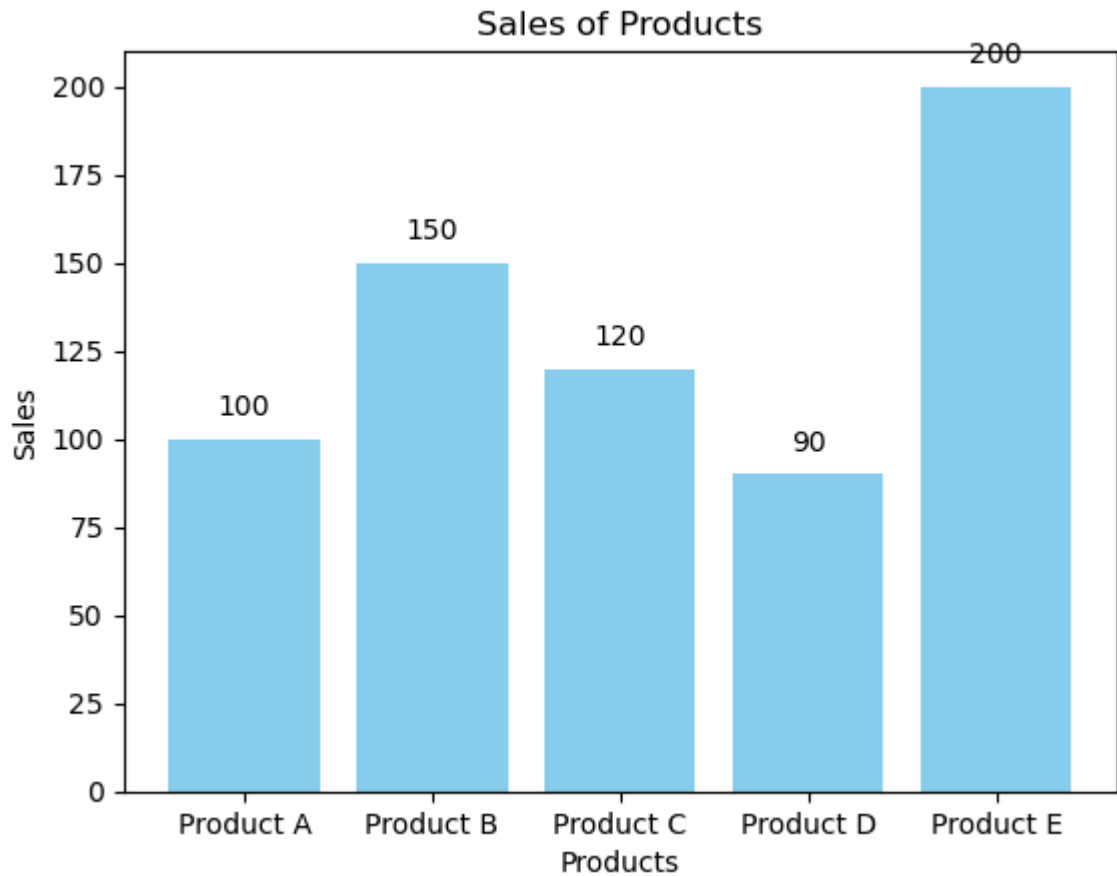
```
In [27]: r.seed(1)
data = [r.randint(1,20) for i in range(20)]
plt.hist(data, bins=5, color='skyblue', edgecolor='black')
plt.title('Histogram of Data')
plt.xlabel('Data Ranges')
plt.ylabel('Frequency')
plt.show()
```



08) WAP to display the value of each bar in a bar chart using Matplotlib.

```
In [31]: products = ['Product A', 'Product B', 'Product C', 'Product D', 'Product E']
sales = [100, 150, 120, 90, 200]
plt.bar(products, sales, color='skyblue')
for i in range(len(products)):
    plt.text(i, sales[i] + 5, str(sales[i]), ha='center', va='bottom', fontsize=12)

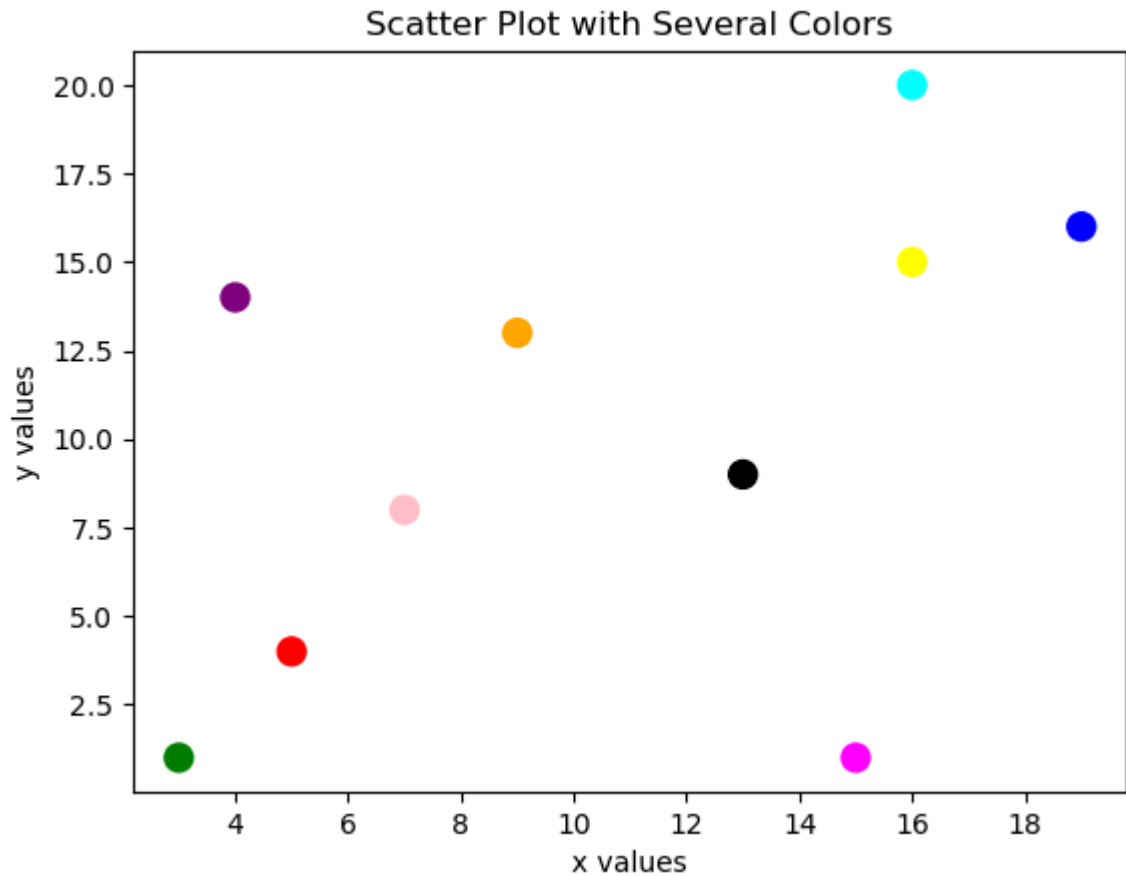
plt.title('Sales of Products')
plt.xlabel('Products')
plt.ylabel('Sales')
plt.show()
```

09) WAP create a Scatter Plot with several colors in Matplotlib?

```
In [31]: r.seed(1)
x = [r.randint(1,20) for i in range(10)]
y = [r.randint(1,20) for i in range(10)]
colors = ['red', 'blue', 'green', 'orange', 'purple', 'cyan', 'magenta', 'yellow']

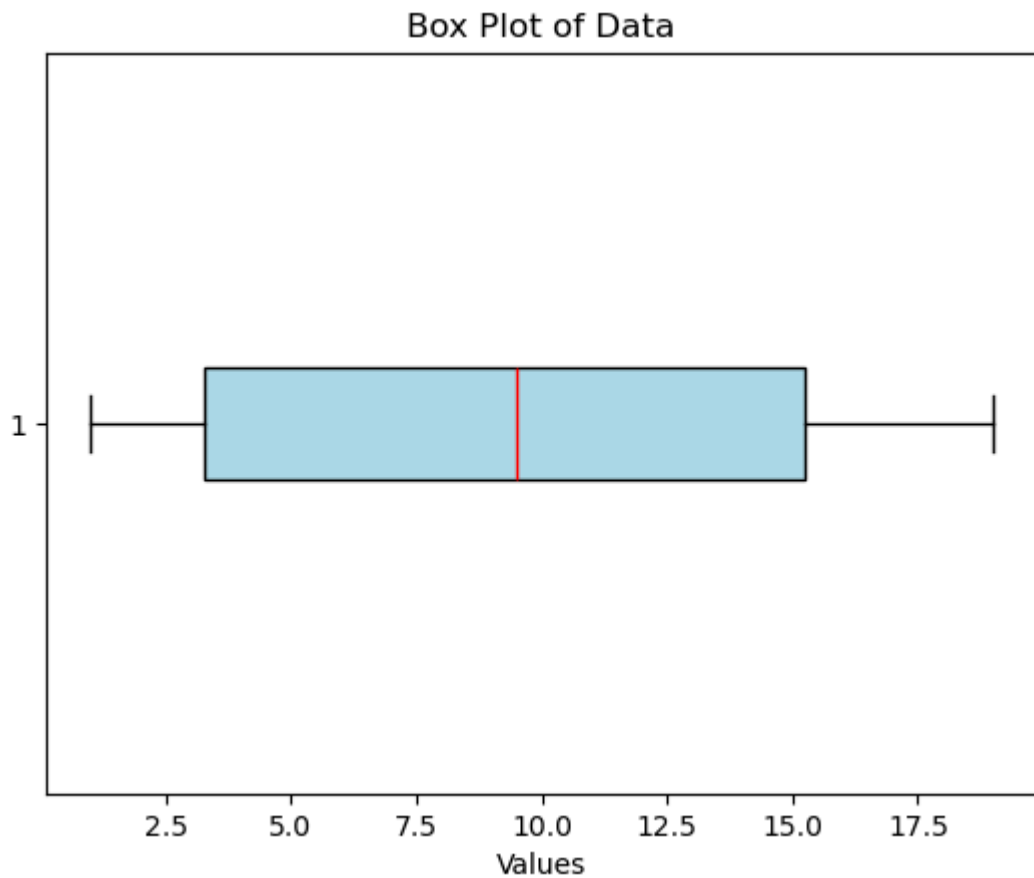
plt.scatter(x, y, color=colors, s=100)
plt.title('Scatter Plot with Several Colors')
plt.xlabel('x values')
plt.ylabel('y values')
plt.show()
```



10) WAP to create a Box Plot.

```
In [33]: data = [r.randint(1,20) for i in range(20)]

plt.boxplot(data, vert=False, patch_artist=True, boxprops=dict(facecolor='lightb
plt.title('Box Plot of Data')
plt.xlabel('Values')
plt.show()
```



Donut Chart

```
In [26]: plt.pie([1, 2, 3, 4, 5])  
plt.pie([1], radius=0.7, colors='w')  
plt.show()
```



```
In [ ]:
```




Python Programming - 2301CS404

Lab - 13_1

Kalp Viradia | 23010101299 |
03/03/2025

OOP

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

```
In [1]: class Student:
        def __init__(self, name, age, grade):
            self.name = name
            self.age = age
            self.grade = grade

        def display_info(self):
            print(f"Name: {self.name}")
            print(f"Age: {self.age}")
            print(f"Grade: {self.grade}")

        student1 = Student("Rahul", 16, "10th Grade")
        student1.display_info()
```

Name: Rahul
Age: 16
Grade: 10th Grade

02) Create a class named Bank_Account with Account_No, User_Name, Email, Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

```
In [5]: class Bank_Account:
    def GetAccountDetails(self, account_no, user_name, email, account_type, account_balance):
        self.account_no = account_no
        self.user_name = user_name
        self.email = email
        self.account_type = account_type
        self.account_balance = account_balance

    def DisplayAccountDetails(self):
        print("Account Details:")
        print(f"Account No: {self.account_no}")
        print(f"User Name: {self.user_name}")
        print(f>Email: {self.email}")
        print(f"Account Type: {self.account_type}")
        print(f"Account Balance: ${self.account_balance}")

account1 = Bank_Account()
account1.GetAccountDetails(123456, "Rahul", "rahul@gmail.com", "Savings", 50000)
account1.DisplayAccountDetails()
```

Account Details:
 Account No: 123456
 User Name: Rahul
 Email: rahul@gmail.com
 Account Type: Savings
 Account Balance: \$50000

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

```
In [9]: import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * (self.radius ** 2)

    def perimeter(self):
        return 2 * math.pi * self.radius

circle = Circle(5)
area = circle.area()
print(f"Area of the circle: {area:.2f}")
perimeter = circle.perimeter()
print(f"Perimeter (Circumference) of the circle: {perimeter:.2f}")
```

Area of the circle: 78.54
 Perimeter (Circumference) of the circle: 31.42

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

```
In [13]: class Employee:
    def __init__(self, name, age, salary):
        self.name = name
```

```

        self.age = age
        self.salary = salary

    def update_info(self, name=None, age=None, salary=None):
        if name:
            self.name = name
        if age:
            self.age = age
        if salary:
            self.salary = salary

    def display_info(self):
        print("Employee Information:")
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Salary: ${self.salary}")

emp = Employee("Rahul", 30, 50000)
emp.display_info()
emp.update_info(name="Ravi", age=32, salary=55000)
emp.display_info()

```

Employee Information:
 Name: Rahul
 Age: 30
 Salary: \$50000
 Employee Information:
 Name: Ravi
 Age: 32
 Salary: \$55000

05) Create a bank account class with methods to deposit, withdraw, and check balance.

```

In [15]: class BankAccount:
    def __init__(self, account_number, account_holder, balance=0):
        self.account_number = account_number
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited ${amount}. Current balance: ${self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew ${amount}. Current balance: ${self.balance}")
            else:
                print("Insufficient funds.")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        print(f"Current balance: ${self.balance}")

```

```

account = BankAccount(123456, "Rahul", 1000)
account.deposit(500)
account.withdraw(200)
account.check_balance()
account.withdraw(1500)

```

Deposited \$500. Current balance: \$1500

Withdrew \$200. Current balance: \$1300

Current balance: \$1300

Insufficient funds.

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

```

In [17]: class Inventory:
    def __init__(self):
        self.items = {}

    def add_item(self, item_name, price, quantity):
        if item_name in self.items:
            self.items[item_name]['quantity'] += quantity
        else:
            self.items[item_name] = {'price': price, 'quantity': quantity}
        print(f"Added {quantity} {item_name}(s) to inventory.")

    def remove_item(self, item_name, quantity):
        if item_name in self.items:
            if self.items[item_name]['quantity'] >= quantity:
                self.items[item_name]['quantity'] -= quantity
                print(f"Removed {quantity} {item_name}(s) from inventory.")
            else:
                print(f"Not enough {item_name} in inventory to remove.")
        else:
            print(f"{item_name} not found in inventory.")

    def update_price(self, item_name, new_price):
        if item_name in self.items:
            self.items[item_name]['price'] = new_price
            print(f"Updated price of {item_name} to ${new_price}.")
        else:
            print(f"{item_name} not found in inventory.")

    def display_inventory(self):
        if not self.items:
            print("Inventory is empty.")
        else:
            print("Inventory Details:")
            for item_name, details in self.items.items():
                print(f"{item_name}: Price - ${details['price']}, Quantity - {de

inventory = Inventory()
inventory.add_item("Laptop", 1200, 10)
inventory.add_item("Phone", 800, 15)
inventory.display_inventory()
inventory.remove_item("Laptop", 5)
inventory.update_price("Phone", 850)
inventory.display_inventory()

```



```
inventory.remove_item("Laptop", 6)
inventory.update_price("Tablet", 300)
```

Added 10 Laptop(s) to inventory.
 Added 15 Phone(s) to inventory.
 Inventory Details:
 Laptop: Price - \$1200, Quantity - 10
 Phone: Price - \$800, Quantity - 15
 Removed 5 Laptop(s) from inventory.
 Updated price of Phone to \$850.
 Inventory Details:
 Laptop: Price - \$1200, Quantity - 5
 Phone: Price - \$850, Quantity - 15
 Not enough Laptop in inventory to remove.
 Tablet not found in inventory.

07) Create a Class with instance attributes of your choice.

```
In [19]: class Car:
    def __init__(self, make, model, year, color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color

    def display_car_info(self):
        print(f"Car Information:")
        print(f"Make: {self.make}")
        print(f"Model: {self.model}")
        print(f"Year: {self.year}")
        print(f"Color: {self.color}")

car1 = Car("Toyota", "Corolla", 2021, "Blue")
car1.display_car_info()
car2 = Car("Honda", "Civic", 2020, "Red")
car2.display_car_info()
```

Car Information:
 Make: Toyota
 Model: Corolla
 Year: 2021
 Color: Blue
 Car Information:
 Make: Honda
 Model: Civic
 Year: 2020
 Color: Red

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

```
In [23]: class StudentKit:
    principal = "Mr ABC"

    def __init__(self, student_name):
        self.student_name = student_name
        self.attendance = 0

    def record_attendance(self, days_present):
        self.attendance = days_present
        print(f"Attendance recorded: {self.attendance} days")

    def generate_certificate(self):
        if self.attendance >= 75:
            print(f"Certificate of Attendance\n")
            print(f"Principal: {StudentKit.principal}")
            print(f"Student: {self.student_name}")
            print(f"Days Present: {self.attendance}")
            print(f"Status: Passed (Attendance is sufficient)")
        else:
            print(f"Certificate of Attendance\n")
            print(f"Principal: {StudentKit.principal}")
            print(f"Student: {self.student_name}")
            print(f"Days Present: {self.attendance}")
            print(f"Status: Failed (Attendance is insufficient)")

student_name = input("Enter the student's name: ")
student = StudentKit(student_name)
days_present = int(input("Enter the number of days present in class: "))
student.record_attendance(days_present)
student.generate_certificate()
```

Attendance recorded: 98 days
Certificate of Attendance

Principal: Mr ABC
Student: Kalp
Days Present: 98
Status: Passed (Attendance is sufficient)

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

```
In [25]: class Time:
    def __init__(self, hour=0, minute=0):
        self.hour = hour
        self.minute = minute

    def add_time(self, other):
        total_minutes = (self.hour * 60 + self.minute) + (other.hour * 60 + other.minute)
        total_hour = total_minutes // 60
        total_minute = total_minutes % 60
        return Time(total_hour, total_minute)

    def display_time(self):
        print(f"{self.hour} hour(s) and {self.minute} minute(s)")

time1 = Time(2, 45)
```

```
time2 = Time(3, 30)
total_time = time1.add_time(time2)
print("Time 1:")
time1.display_time()
print("Time 2:")
time2.display_time()
print("Total Time after adding:")
total_time.display_time()
```

Time 1:
2 hour(s) and 45 minute(s)
Time 2:
3 hour(s) and 30 minute(s)
Total Time after adding:
6 hour(s) and 15 minute(s)

In []:



Python Programming - 2301CS404

Lab - 13_2

Kalp Viradia | 23010101299 |
10/03/2025

Continued..

10) Calculate area of a rectangle using object as an argument to a method.

```
In [2]: class Rectangle:
        def __init__(self, length, width):
            self.length = length
            self.width = width

        def calculate_area(self):
            return self.length * self.width

        rect = Rectangle(10, 5)

        def get_area(rectangle):
            return rectangle.calculate_area()

        area = get_area(rect)
        print("Area of rectangle:", area)
```

Area of rectangle: 50

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

```
In [4]: class Square:
        def __init__(self, side_length):
```

```

        self.side_length = side_length

    def area(self):
        area = self.side_length ** 2
        self.output(area)

    def output(self, area):
        print(f"The area of the square is: {area}")

square = Square(4)
square.area()

```

The area of the square is: 16

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named `area()` and a method named `output()` that prints the output and is invoked by `area()`.

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

```

In [6]: class Rectangle:
    def __init__(self, length, width):
        if length == width:
            print("THIS IS SQUARE")
            self.length = None
            self.width = None
        else:
            self.length = length
            self.width = width

    def area(self):
        if self.length and self.width:
            area = self.length * self.width
            self.output(area)
        else:
            print("Cannot calculate area for a square object.")

    def output(self, area):
        print(f"The area of the rectangle is: {area}")

    @classmethod
    def compare_sides(cls, length, width):
        if length == width:
            print("THIS IS SQUARE")
        else:
            print("This is a rectangle.")

rect1 = Rectangle(10, 5)

if rect1.length and rect1.width:
    rect1.area()

```

```
Rectangle.compare_sides(10, 5)
Rectangle.compare_sides(5, 5)
```

The area of the rectangle is: 50
This is a rectangle.
THIS IS SQUARE

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to access the private attribute from outside of the class.

```
In [8]: class Square:
        def __init__(self, side):
            self.__side = side

        def get_side(self):
            return self.__side

        def set_side(self, side):
            if side > 0:
                self.__side = side
            else:
                print("Side length must be a positive number!")

        def area(self):
            return self.__side ** 2

square = Square(5)
print("Side of square:", square.get_side())
square.set_side(10)
print("New side of square:", square.get_side())
print("Area of square:", square.area())
```

Side of square: 5
New side of square: 10
Area of square: 100

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balance. It has two methods getBalance() and printBalance().

```
In [10]: class Profit:
        def __init__(self):
            self.profit = 0

        def getProfit(self):
            self.profit = float(input("Enter profit amount: "))

class Loss:
```

```

def __init__(self):
    self.loss = 0

def getLoss(self):
    self.loss = float(input("Enter loss amount: "))

class BalanceSheet(Profit, Loss):
    def __init__(self):
        Profit.__init__(self)
        Loss.__init__(self)
        self.balance = 0

    def getBalance(self):
        self.balance = self.profit - self.loss

    def printBalance(self):
        print(f"Profit: {self.profit}")
        print(f"Loss: {self.loss}")
        print(f"Balance: {self.balance}")

balance_sheet = BalanceSheet()
balance_sheet.getProfit()
balance_sheet.getLoss()
balance_sheet.getBalance()
balance_sheet.printBalance()

```

Profit: 500.0
 Loss: 100.0
 Balance: 400.0

15) WAP to demonstrate all types of inheritance.

```

In [12]: # Single Inheritance
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

# Multiple Inheritance
class Person:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f"Hello, my name is {self.name}")

class Employee:
    def __init__(self, employee_id):
        self.employee_id = employee_id

    def show_id(self):
        print(f"My employee ID is {self.employee_id}")

class Manager(Person, Employee):
    def __init__(self, name, employee_id):
        Person.__init__(self, name)

```

```

        Employee.__init__(self, employee_id)

    def work(self):
        print(f"{self.name} is working as a Manager.")

# Multilevel Inheritance
class Vehicle:
    def start_engine(self):
        print("Vehicle engine started")

class Car(Vehicle):
    def drive(self):
        print("Car is driving")

class SportsCar(Car):
    def speed(self):
        print("Sports car is speeding")

# Hierarchical Inheritance
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def area(self, radius):
        return 3.14 * radius * radius

class Rectangle(Shape):
    def area(self, length, width):
        return length * width

# Hybrid Inheritance (Combination of Multiple and Multilevel Inheritance)
class School:
    def __init__(self, name):
        self.name = name

    def show_name(self):
        print(f"School Name: {self.name}")

class Teacher(School):
    def __init__(self, name, subject):
        School.__init__(self, name)
        self.subject = subject

    def teach(self):
        print(f"Teaching {self.subject}")

class HeadTeacher(Teacher):
    def __init__(self, name, subject, head_teacher_name):
        Teacher.__init__(self, name, subject)
        self.head_teacher_name = head_teacher_name

    def manage(self):
        print(f"{self.head_teacher_name} manages the teaching process.")

# 1. Single Inheritance
print("Single Inheritance:")
dog = Dog()
dog.speak()
dog.bark()

```



```

print()

# 2. Multiple Inheritance
print("Multiple Inheritance:")
manager = Manager("ABC", 101)
manager.introduce()
manager.show_id()
manager.work()
print()

# 3. Multilevel Inheritance
print("Multilevel Inheritance:")
sports_car = SportsCar()
sports_car.start_engine()
sports_car.drive()
sports_car.speed()
print()

# 4. Hierarchical Inheritance
print("Hierarchical Inheritance:")
circle = Circle()
print(f"Circle Area: {circle.area(5)}")
rectangle = Rectangle()
print(f"Rectangle Area: {rectangle.area(10, 5)}")
print()

# 5. Hybrid Inheritance
print("Hybrid Inheritance:")
head_teacher = HeadTeacher("DEF", "Maths", "ABC")
head_teacher.show_name()
head_teacher.teach()
head_teacher.manage()

```

Single Inheritance:

Animal speaks

Dog barks

Multiple Inheritance:

Hello, my name is ABC

My employee ID is 101

ABC is working as a Manager.

Multilevel Inheritance:

Vehicle engine started

Car is driving

Sports car is speeding

Hierarchical Inheritance:

Circle Area: 78.5

Rectangle Area: 50

Hybrid Inheritance:

School Name: DEF

Teaching Maths

ABC manages the teaching process.

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class **Employee** that inherits from **Person** and adds a new attribute **salary**.

Override the **init** method in **Employee** to call the parent class's **init** method using the **super()** and then initialize the **salary** attribute.

```
In [14]: class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_info(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")

class Employee(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.salary = salary

    def display_employee_info(self):
        self.display_info()
        print(f"Salary: {self.salary}")

emp = Employee("ABC", 30, 50000)
emp.display_employee_info()
```

Name: ABC

Age: 30

Salary: 50000

17) Create a Shape class with a draw method that is not implemented.

Create three child classes **Rectangle**, **Circle**, and **Triangle** that implement the **draw** method with their respective drawing behaviors.

Create a list of **Shape** objects that includes one instance of each child class, and then iterate through the list and call the **draw** method on each object.

```
In [16]: from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def draw(self):
        pass

class Rectangle(Shape):
    def draw(self):
        print("Drawing a rectangle")

class Circle(Shape):
```

```
def draw(self):  
    print("Drawing a circle")  
  
class Triangle(Shape):  
    def draw(self):  
        print("Drawing a triangle")  
  
shapes = [Rectangle(), Circle(), Triangle()]  
  
for shape in shapes:  
    shape.draw()
```

Drawing a rectangle

Drawing a circle

Drawing a triangle

In []: